

微网与外部电网电力调控策略

摘要

对于问题一, 建立带周期库存约束的确定性线性规划: 在母线侧能量平衡、光伏消纳与弃光、储能状态与功率边界约束下, 最小化全天计划购电费用。求解得全天购电费 35126.949 元、购电量 59482.699 kWh, 较无储能基线降低 26.8981%。独立逐槽复算与显式互斥 MILP 对拍结果一致, 结果在本文约定口径下可信。

对于问题二, 在问题一内核上建立严格因果的全年逐日滚动模型: 每天 0:00 仅用该时刻已观测到的历史数据, 以七天前同刻点预测当天负荷与光伏, 并以滚动 7 天视野协同优化、只执行当天计划; 储能储电量跨日连续传递, 供电缺口按交易时刻电价的 5 倍紧急购电, 未使用的合同电按计划量计费但不进入母线。对 2025 年 2 月 1 日至 12 月 31 日共 334 天顺序回放, 全年购电费 16 886 077.67 元 (正常合同电费 12 190 821.03 元、紧急购电费 4 695 256.64 元), 紧急购电量 1 199 863.45 kWh, 334 天全部可行, 跨日储电量连续无断裂。同年度回放中 D-7 费用低于同星期日衰减平均; 另测得的联合分位扰动候选费用更高, 但该结果不否定其他稳健优化设计。

对于问题三, 在问题二的因果回放框架上建立四时点确定性模型预测控制模型。附件 3 的整点光伏点预测经无泄漏线性展开后, 在 0:00、6:00、12:00 和 18:00 依次更新; 已执行时段锁定, 未执行时段按相邻版本的非对称现金流重优化。334 天回放总费用为 15805751.77 元, 较问题二降低 6.3977% (其中仅约 3.23 个百分点来自追加日内更新), 紧急购电量降低 32.5567%。其中换用附件 3 的 0:00 预报及相应计划降低费用 53.54 万元, 追加 6:00、12:00、18:00 更新再降低 54.49 万元; 物理约束和费用账本均经独立复算。

对于问题四, 将附件 4 的 10 分钟波动电价接入问题二、三的滚动框架, 并严格分离“决策时可得的预测价”和“事后结算的真实价”。仅用一月顺序窗口选定同星期日四周衰减加权预测。第二问式与第三问式全年实际费用分别为 17 743 324.88 元和 16 606 095.44 元; 更换光伏预报来源并加入日内更新后的合并效果为节省 1 137 229.44 元, 降幅 6.4093%, 紧急购电量降低 32.7207%。逐槽物理量、真实价格和版本账本均经独立复算。

四问共用同一储能物理内核、因果信息边界与独立核算链; 完美信息下界用于量化负荷、光伏与电价联合信息的价值, 不参与正式计划。

关键词: 微网; 储能调度; 线性规划; 购电策略; 光伏消纳

一、引言

1.1 问题背景

小区级微网通过协调屋顶光伏、储能设备与外部电网，实现局部供需平衡和经济运行；储能则是提高新能源消纳能力与调度灵活性的关键环节 [1]。光伏出力由日照决定，夜间为零、正午达到峰值；小区负荷则呈现早起与傍晚双高峰的日周期变化，两者在时间上并不同步。这一错配使微网在无储能时只能“光伏富余时弃光、负荷高峰时高价购电”，既浪费了本地新能源，又抬高了用电成本。储能设备把这部分时间差转化为套利空间：在光伏富余或电价低谷时充电，在电价高峰时放电，从而在满足负荷的前提下压低全天购电费用。

问题所给场景的时间粒度为 10 分钟，一天划分为 144 个时段，决策变量是每个时段的购电量和储能充放电量。外部电网采用分时电价：夜间低谷电价约为峰段的三分之一，正午前后又因光伏集中出力而回落。储能设备最大容量 12000 kWh，允许荷电状态 (State of Charge, SOC) 区间为 1200 至 10800 kWh，最大充放电功率 5000 kW，充放电效率均为 90%，初始电量 6000 kWh。这些参数决定了模型必须在物理边界、功率上限和周期库存之间取得平衡。

四问在同一物理内核上逐级加难。问题一给定单日完整的分时电价、负荷与光伏预测，属于确定性调度；问题二把场景扩展到全年并引入预测，负荷与光伏只能由历史数据推得，供电缺口须按交易时刻电价的 5 倍紧急购电；问题三在日内 0:00、6:00、12:00、18:00 追加已发布的光伏预报，允许对未执行时段调整计划，并按上调 1.5 倍、下调 0.5 倍结算；问题四把固定日内电价替换为逐 10 分钟波动的真实电价，重新评估前述两套机制。四问的关系是“同一套储能与能量平衡约束，叠加不同的信息集与结算规则”。

1.2 研究意义

本文的处理对象是一个“信息受限下的滚动优化”问题，其意义体现在三个层面。

在经济层面，储能调度的正确性可以用统一的费用账本量化。分时电价和波动电价下的购电决策直接决定微网运行成本，而 5 倍紧急购电价把预测偏差的代价放大到正常电价的数倍。若在决策时误用未来真实数据，紧急购电会被压至零，得到看似优异却不可执行的调度。本文坚持严格因果的信息边界，使得到的费用是可执行的、可结算的，并可用完美信息下界度量预测信息的价值。

在方法层面，本文把“预测—优化—执行—核算”拆成相互独立的模块，并让每一份历史数据与预报都记录其可观测时刻。这一设计使任何一次决策都能被追溯审计：优化器只看到决策时刻及之前的数据，核算器不调用优化器的目标值而是从逐槽记录独立复算。该结构适用于储能容量、分时电价、光伏渗透率不同的同类微网，也便于在引入新信息源（更密的预报、外生特征）时定位收益来源。

在赛题层面，本文正面回答题目要求的边界判断：问题三需要判断是否引入其他时刻的光伏预报，问题四需要区分”事前可得的预测价”与”事后结算的真实价”。这类问题的答案不由预测误差单独决定，而须由扣除调整与违约成本后的全年实际费用给出，本文据此给出可复现的数值结论。

1.3 问题重述

问题 1：已知某天的分时电价、小区负载与光伏发电功率预测，要求微网供电不低于负载且储能设备在 0:00 与 24:00 的储电量相同，在每天 0:00 制定当天的计划购电策略，并在论文中给出指定时间段的购电量、充放电量及全天购电量与购电费。

问题 2：在每天电价相同、小区负载与光伏发电功率随时间变化的条件下，微网供电不可低于负载，低于负载时需按交易时刻电价的 5 倍向外网紧急购电，除紧急购电外其余时段均按每天 0:00 制定的计划购电量计费。要求每天 0:00 制定当天 144 个时段的计划购电策略，给出 2025 年 2 月 1 日至 12 月 31 日的完整计划购电量、储能充放电量及紧急购电的时间段与电量。

问题 3：在问题二的基础上，利用每天 0:00、6:00、12:00 和 18:00 发布的未来 24 小时光伏预测，对尚未执行时段滚动调整购电策略；按上调 1.5 倍、下调 0.5 倍和紧急购电 5 倍的规则核算费用，给出全年计划购电量、调整购电量、充放电量与紧急购电量，并判断是否需要引入其他时刻的预报。

问题 4：将外网电价改为附件 4 给出的 10 分钟波动价格，分别沿用问题二的每日计划机制和问题三的日内调整机制，重新计算 2025 年 2 月 1 日至 12 月 31 日的计划购电、储能充放电与紧急购电结果，并填写 `result4-2` 和 `result4-3`。

二、问题分析

四问共用一个含光伏与储能的微网，在满足负荷的前提下最小化购电费用，差异在于决策时可得的信息与结算规则。下文先说明共用模型，再逐问说明新增约束与处理方法。

2.1 总体建模思路

储能调度的本质是”把电量在时间上搬移”。在每个 10 分钟时段，能量的来源有购电、光伏消纳和放电三项，去向有负荷、充电和弃光三项，母线侧平衡关系固定。储能状态按其充放电效率逐时段递推，并受容量与功率边界约束。四问的目标函数都是购电费用，只是费用系数和可用信息不同。

据此，本文不逐问另建模型，而是固定一个确定性线性规划内核：约束为母线能量平衡、光伏消纳、储能状态递推、容量与功率边界，决策变量为购电量与充放电量。微

网经济调度通常可表述为满足功率平衡、储能状态和设备运行约束下的优化问题，线性规划与混合整数线性规划是该类问题常用的建模方法 [2, 3]。问题一在这个内核上直接求解；问题二至问题四只替换目标函数中的电价、替换输入序列的获得方式（由真值改为预测），并在其上叠加执行与结算规则。这样做避免了四套模型口径互不一致，也使计算结果能够被同一套核算器复算。

方法选择上，之所以采用线性规划而非启发式或强化学习：其一，约束（能量平衡、状态递推、功率上限）全部线性，问题规模（单日 144 槽、视野 7 天）在现代求解器上可精确求解，无需引入近似；其二，官方要求结果可复现且逐槽可核对，线性规划的最优解与对偶信息都能给出可验证的边界，便于用对拍模型交叉验证；其三，问题三的日内更新与问题四的价格替换都是在同一线性结构上改动目标系数，改动量小、可审计。预测部分之所以采用历史同刻类基线而非复杂时间序列模型，是因为评价标准是实际费用而非预测误差，简单可审计的预测器更容易定位收益来源，也避免在全年数据上拟合后再声称顺序回测。

2.2 问题一的分析

问题一的核心难点是终端库存与套利动机的冲突。若只要求全天购电费用最小，模型会在期末把储电量放到最低（1200 kWh），用耗空电池换取当天的低成本，但这并非可循环的日计划。题面明确要求 0:00 与 24:00 的储电量相同，因此终端库存必须回到初始值，储能的价值只能来自日内时间差，不能来自跨日透支。

在该约束下，问题的数学对象是一个带周期库存约束的确定性线性规划：决策变量是 144 个时段的购电量与充放电量，目标是最小化 $\sum_t p_t g_t$ ，约束包括母线平衡、光伏消纳与弃光、状态递推、容量与功率边界，以及周期库存等式。计划购电、负荷与光伏均为确定性输入，不涉及预测，也不需要处理紧急购电与调整结算。本文采用线性规划精确求解，并另建显式充放电互斥的混合整数模型作对拍，用能量残差、约束违反次数和替代模型费用差验证结果。

2.3 问题二的分析

问题二把单日确定性调度扩展为全年逐日滚动决策，新增三类耦合，构成核心难点。

其一是信息约束。每天 0:00 制定计划时，当天真实负荷与光伏尚未发生，只能使用历史数据，因此必须先建立严格因果的负荷与光伏预测器，任何一次决策只允许读取决策时刻及之前已观测的数据。

其二是跨日状态约束。储能储电量在相邻日之间必须连续传递，若每天把初值重置为 6000 kWh，会虚构能量并扭曲储能价值，因此需明确零点衔接规则，用旧计划末槽动作与当前实际储电量估计新计划的起点。

其三是终端库存约束。在本文相同预测输入与执行口径的对照回放中，每天只优化当天并强制首尾储电量相等的候选出现日末库存持续停在容量上限的现象；若完全不约束期末库存，模型又会透支电池。因此本文引入跨日前瞻机制约束期末储电量。该现象仅是本实验条件下的结果，不外推为所有日周期策略的必然性质。

此外，供电缺口须按交易时刻电价的 5 倍紧急购电，标称价格的保守计划可能反而更贵。本文采用滚动 7 天视野的逐日线性规划：因果预测、按日滚动、只执行当天计划，并以滚动视野的库存回归约束替代对期末库存的人为定价。这种“有限视野求解、执行首段、获得新信息后再求解”的结构与模型预测控制一致 [4]，全部结果由独立核算器逐槽复算。

2.4 问题三的分析

问题三的新增信息不是当天光伏真值，而是日内 0:00、6:00、12:00、18:00 逐次发布的未来 24 小时光伏预报。因此核心难点不是重建储能模型，而是在问题二的执行器上加入预测发布、计划改版、按版结算的闭环：预报更新可以减少预测偏差引起的紧急购电，但频繁改版又会产生上调溢价与下调冲回，两者存在权衡。

问题还要求判断是否需要引入其他时刻的预报。该判断不能只比较预报误差，必须由扣除调整成本后的全年实际费用、紧急购电量和调整规模共同决定。已有研究已将模型预测控制用于含储能微网的运行优化 [5]。本文据此采用多时点模型预测控制：以问题二物理执行器为基础，在每次发布时点用最新预报对未执行时段重新优化，已执行时段锁定，按相邻版本的调整现金流定价，并通过逐级加入 6:00、12:00、18:00 批次的消融实验回答是否需要更多预报。

2.5 问题四的分析

问题四不改变储能物理结构，也不新增可控设备，变化在于外网电价由固定日内曲线改为逐 10 分钟波动曲线。核心难点是价格不确定性下的信息边界：若把附件 4 的未来真实价格直接送入优化器，虽然费用更低，却等于在 0:00 提前知道当天乃至后续价格，属于信息泄漏；若只用历史价格，又必须承受价格预测误差带来的成本。

本文采用双价格通道处理该难点：预测价只进入计划优化，真实价只进入执行回放与费用结算，并另设使用未来真实价格的完美信息模型作为不可执行的下界对照。具体做法是在问题二、问题三的滚动框架上分别替换价格层，得到第二问式与第三问式两条策略，用同一核算器在同一指标下比较其全年实际费用，并用下界量化价格预测信息不足所造成的代价。

三、模型假设

为简化问题，在不影响主要结论的前提下，作如下假设：

1. 微网系统结构及设备参数在研究周期内保持不变，各设备均正常运行，不考虑故障及通信异常。
2. 微网仅从外部电网购电，不向外网反送电；当光伏出力超过负荷需求和储能接纳能力时，允许弃光。
3. 储能充、放电效率及容量、功率约束保持不变，储能状态在相邻时段及相邻日期间连续传递。
4. 负荷、光伏、电价等变量均按题目给定时间尺度离散，同一时间段内相关参数视为恒定，并忽略题目未给出的线路及附加能量损耗。
5. 所有预测与调度决策仅使用决策时刻及之前可获得的信息，不使用未来真实数据；题目所给历史及预测数据视为真实有效。

四、符号说明

本节只列出跨问通用的核心物理符号；各问局部符号在其首次出现处就地定义。表中能量单位为 kWh，功率单位为 kW，电价单位为元/kWh。

表 1 全文主要符号

符号	含义
t	10 分钟时段序号， $t = 0, 1, \dots, 143$
Δt	时段长度， $\Delta t = 1/6 \text{ h}$
p_t	时段 t 电价
L_t	时段 t 负荷电量
V_t	时段 t 可用光伏电量
v_t	时段 t 实际消纳光伏电量
q_t^{curt}	时段 t 弃光电量
g_t	时段 t 外网计划购电量
c_t, d_t	母线侧充电量、放电电量
E_t	时段 t 边界储电量
E^{max}	储电量允许上限，取 10800
η_c, η_d	充、放电效率，均取 0.9
g_t^{emg}	时段 t 紧急购电量，按 $5p_t$ 计费
C	全天购电费用

五、问题一的模型的建立和求解

5.1 具体分析

问题一给出了一个典型日内完整的分时电价、小区负荷和光伏发电预测，因此不需要再建立预测模型，而是要在满足负荷与储能约束的前提下，确定 144 个 10 分钟时段的计划购电量及储能充放电量。该问题的核心是利用低价电和光伏富余给电池充电，在高价时段放电，同时避免通过期末耗空电池获得虚假的低成本。由此可将问题建立为带周期库存约束的确定性线性规划。

5.2 数据预处理：时间尺度与索引定义

本节定义四问共用的时间索引、单位与数据读取口径。令 $t = 0, 1, \dots, 143$ 表示 10 分钟时段， $\Delta t = 1/6$ h。按本文约定的时间映射（方案 A）：附件数据、模型槽位和结果模板按位置一一对应、不平移，槽 0 对应时段 0:10-0:20，槽 142 对应 23:50-24:00，槽 143 对应次日 0:00-0:10；附件缺少当天 0:00-0:10 的输入，该段不单独调度且储电量不变。因此 E_0 对应当天 0:10， E_{143} 对应次日 0:00（当天 24:00）， E_{144} 对应次日 0:10。附件 1 中负荷和光伏为功率量，对每个时段分别计算电量

$$L_t = P_t^{\text{load}} \Delta t, \quad V_t = P_t^{\text{pv}} \Delta t. \quad (1)$$

读取时要求价格、负荷和光伏恰有 144 个数值，无缺失和重复；时间映射统一由槽位编号完成，不使用模糊的时间字符串连接。充电量和放电量均按交流母线侧计量，充、放电效率分别取 0.9；光伏富余且储能满充时允许弃光（ $q_t^{\text{curt}} \geq 0$ ），购电量满足 $g_t \geq 0$ 。

5.3 问题一模型设定

问题一采用确定性输入：每天 0:00 制定计划时，附件 1 中的当日电价、负荷和光伏预测均已知，且计划执行期间不再调整，因此不产生紧急购电、未使用合同电和调整结算费用。每个时段的母线侧能量平衡为

$$g_t + v_t + d_t = L_t + c_t, \quad (2)$$

其中 g_t 为外网计划购电量， v_t 为实际消纳光伏电量， c_t, d_t 为母线侧充电量、放电量。光伏消纳及弃光满足

$$0 \leq v_t \leq V_t, \quad q_t^{\text{curt}} = V_t - v_t \geq 0. \quad (3)$$

储能状态按充放电效率递推：

$$E_{t+1} = E_t + \eta_c c_t - \frac{d_t}{\eta_d}, \quad \eta_c = \eta_d = 0.9. \quad (4)$$

容量、功率和购电约束为

$$1200 \leq E_t \leq E^{\max}, \quad 0 \leq c_t, d_t \leq 5000\Delta t = 833.333, \quad g_t \geq 0, \quad (5)$$

其中 $E^{\max} = 10800 \text{ kWh}$ 为储电量允许上限。记模型首槽起点库存为 E_0 、144 个模板槽执行后的库存为 E_{144} 。冻结模型施加的是模型周期等式

$$E_0 = E_{144} = 6000. \quad (6)$$

主目标为最小化全天正常购电费用 C :

$$\min C = \sum_{t=0}^{143} p_t g_t. \quad (7)$$

由于线性规划可能存在多个同费用解，先求得最优费用 C^* ，再在 $C \leq C^* + 10^{-7}$ 的条件下最小化 $\sum_t (c_t + d_t)$ 。该字典序目标只消除无意义的充放电循环，不改变经济目标。

5.4 模型求解

首先通过 SciPy 优化接口调用 HiGHS dual simplex 求解主模型，并用 HiGHS IPM 对拍目标值；SciPy 为 Python 科学计算提供了包括数值优化在内的基础算法组件 [6]。随后将购电、光伏消纳、充放电和储电量送入独立执行器。核算器不调用优化器目标函数，而是重新检查式 (1) 至式 (6)，逐槽计算 $p_t g_t$ ，最后把审计通过的结果写入官方模板并重新打开回读。论文中的费用均取核算器结果，而不是直接复制求解器日志。

表 2 给出题面指定时段的购电量，完整 144 槽结果保存在随文提交的正式结果文件中。

表 2 问题一指定时间段购电量及全天购电量、购电费

时间段	购电量 (kWh)	时间段	购电量 (kWh)	时间段	购电量 (kWh)
10:00-	0.000	12:00-	486.403	14:00-	0.000
10:10		12:10		14:10	
16:00-	394.931	18:00-	636.983	20:00-	0.000
16:10		18:10		20:10	

全天计划购电量为 59482.699 kWh，购电费为 35126.949 元。光伏消纳量为 55482.836 kWh，弃光量为 0；累计充电量和放电量分别为 20740.666 kWh 与 16799.940 kWh。

表 3 为保持冻结模型与结果模板的逐行对应，按每 24 个槽位汇总充放电量，不另做十分钟平移。表中改用槽位编号，避免将这些位置块误称为严格墙钟区间；例如槽 0-23 实际覆盖 00:10-04:10，并不等同于题面 0:00-4:00。第三问起的指定日期分块改用逐槽真实时间戳聚合，见附录。

表 3 问题一按模板位置分块的储能充放电量

位置 块	充电量 (kWh)	放电量 (kWh)	位置 块	充电量 (kWh)	放电量 (kWh)
槽 0-23	4500.000	0.000	槽 24-47	833.333	6365.841
槽 48-71	4787.964	1702.997	槽 72-95	5286.035	91.101
槽 96-119	0.000	5780.132	槽 120-143	5333.333	2859.868

方案中储电量最小值和最大值分别为 1200 kWh 和 10800 kWh，均位于允许范围内。冻结模型约束的是 $E_0 = E_{144} = 6000$ kWh；在本次求解结果中，槽 142 与槽 143 均无充放电，故 $E_{143} = E_{144} = 6000$ kWh，题面 24:00 对应的 E_{143} 与表中数值恰好一致，但这不是式 (6) 单独保证的性质。与不使用储能的可行基线相比，基线费用为 48052.047 元，本方案减少 12925.098 元，降幅为 26.8981%。这表明储能对分时电价和光伏时移具有明显经济价值。

问题一的模型检验内容见“模型的分析与检验”一节：独立逐槽复算最大残差不超过 10^{-12} 量级，显式互斥 MILP 对拍费用差 7.28×10^{-12} 元，效率口径敏感性约 $\pm 3.8\%$ 。敏感性细节同样在该节给出，此处不再重复。

综上，问题一建立了四问共用的物理和核算内核。确定性 LP 在满足供需平衡、储电量、功率和周期库存约束的情况下，将购电费用降至 35126.949 元，较无储能方案降低 26.8981%。当前主要局限来自题面时间标签和效率语义，而非求解器或结果文件实现。

六、问题二的模型的建立和求解

6.1 具体分析

问题二将问题一的单日确定性调度扩展为全年逐日滚动决策。与问题一相同，每天的电价曲线取自附件 1 的日内曲线并逐日重复；不同之处在于负荷与光伏按附件 2 的实际序列逐日变化，且制定计划时当天的真实值尚未发生。题面规定微网供电不得低于负荷，不足部分必须按交易时刻电价的 5 倍紧急购电，其余时段按当天 0:00 制定的计划购电量计费。由此，问题二的数学对象是一个逐日两阶段决策问题：第一阶段在 0:00 依据截至该时刻的历史信息，确定当天 144 个时段的计划购电量与储能充放电量；第二阶

段在真实负荷与光伏揭示后按计划执行，用紧急购电补足任何供电缺口。

相对于问题一，问题二新增三类耦合，直接决定模型结构。第一是信息约束：计划只能使用决策时刻及之前已观测到的数据，当天真实负荷与光伏不得进入计划，因此必须先建立严格因果的预测器。第二是跨日状态约束：当天计划末槽（次日 0:00 至 0:10）执行后的真实储电量必须作为次日的实际起始状态，逐日重置会虚构能量并扭曲储能价值。第三是终端库存约束：若每天只优化当天，模型会在期末耗空储能换取虚假低价，需要跨日前瞻机制约束期末储电量。三者分别在下文“预测信息集与因果约束”“跨日 SOC 递推”“滚动优化与终端库存策略”三节处理。

6.2 预测信息集与因果约束

每天 0:00 的决策只能使用该时刻及之前已经完整观测到的实际负荷与光伏，即所有历史输入满足

$$\text{observed_at} \leq \text{decision_time}. \quad (8)$$

按时间方案 A，前一天计划的最后一槽位于次日 0:00 至 0:10，要等到 0:10 才观测完整，因此当天 0:00 制定计划时不能读取它的真实值，相应的预测必须回退到更早的历史来源。

负荷取自附件 2 的“小区负载”实测，光伏取自附件 2 的“光伏发电实际功率”实测；功率量按 $\Delta t = 1/6 \text{ h}$ 换算为电量。由于 2 月 1 日的起始储电量需由一月的实际执行结果推出，一月按同一执行器顺序回放作为预热：1 月 1 日没有历史，仅用附件 1 的典型日负荷与光伏作为先验；1 月 2 日至 7 日，每个时段优先采用决策时已经完整观测到的同槽历史均值，尚无观测的时段回退典型日先验；1 月 8 日起启用历史预测基线。2 月 1 日不重置，直接继承 1 月 31 日的实际末储电量。

预测基线采用七天前同一墙钟时刻的实际值作为当天负荷与光伏的点预测，记为 $\hat{L}_t$ 、 $\hat{V}_t$ ，该方法记作 D-7:

$$\hat{L}_t^{(d)} = L_t^{(d-7)}, \quad \hat{V}_t^{(d)} = V_t^{(d-7)}. \quad (9)$$

该基线完全因果、无需拟合参数，能保留完整的周内用电规律。为避免单日噪声与选择偏差，另备同星期日四周衰减平均作为候选，并在全年顺序回放上按实际总费用比较（见下节）。

6.3 费用模型

设时段 t 的计划合同购电量为 g_t ，实际进入母线的合同电为 g_t^{deliv} ，紧急购电量为 g_t^{emg} ，则母线侧能量平衡为

$$g_t^{\text{deliv}} + g_t^{\text{emg}} + v_t + d_t^{\text{act}} = L_t + c_t^{\text{act}}. \quad (10)$$

未实际使用的合同电 $g_t^{\text{unused}} = g_t - g_t^{\text{deliv}}$ 不进入母线，也不额外收费（已含在正常合同电费中）。费用为

$$C = \underbrace{\sum_t p_t g_t}_{\text{正常合同电费}} + \underbrace{\sum_t 5 p_t g_t^{\text{emg}}}_{\text{紧急购电费}}. \quad (11)$$

式 (11) 表明正常电费按计划合同量而非实际受电量计费，多买但未用的电只产生正常电费、不产生额外费用；供电缺口则以紧急购电量 g_t^{emg} 按交易时刻电价的 5 倍另计。

6.4 储能执行与安全削减机制

每个时段储能按充放电效率递推：

$$E_{t+1} = E_t + \eta_c c_t - \frac{d_t}{\eta_d}, \quad \eta_c = \eta_d = 0.9, \quad (12)$$

并满足容量与功率约束

$$1200 \leq E_t \leq 10800, \quad 0 \leq c_t, d_t \leq 5000 \Delta t = 833.333. \quad (13)$$

电池尽量按 0:00 计划动作，不因实际值偏离而实时重调。实际充电量取计划充电量与当前 SOC 剩余充电空间中的较小者；实际放电量取计划放电量、当前实际需求与 SOC 可放电空间中的最小者。计划动作已满足功率上限，执行时只在 SOC 上下界或实际需求不足时进一步削减。削减量逐槽记录为计划值与实际值之差：

$$\Delta c_t^{\text{sf}} = c_t^{\text{plan}} - c_t^{\text{act}}, \quad \Delta d_t^{\text{sf}} = d_t^{\text{plan}} - d_t^{\text{act}}. \quad (14)$$

实际执行遵循“先弃光、再形成未使用合同电、最后削减计划放电”的优先顺序。

6.5 跨日 SOC 递推

0:00 至 0:10 继续执行前一天已经制定的最后一槽计划；新一天的计划在 0:00 制定、从 0:10 开始执行。因此 0:00 制定新计划时，仅用当时已知的实际储电量 $E_0^{(d)}$ 与旧计划已安排的末槽动作 $(c_{143}^{(d-1)}, d_{143}^{(d-1)})$ 估计 0:10 的储电量。为使估计值本身不越过 SOC 边界，先按可用空间削减旧计划动作：

$$\begin{aligned} c_{143}^{\text{br}} &= \min \left(c_{143}^{(d-1)}, \frac{E^{\text{max}} - E_0^{(d)}}{\eta_c} \right), \\ d_{143}^{\text{br}} &= \min \left(d_{143}^{(d-1)}, (E_0^{(d)} - 1200) \eta_d \right), \\ \hat{E}_0^{(d)} &= E_0^{(d)} + \eta_c c_{143}^{\text{br}} - \frac{d_{143}^{\text{br}}}{\eta_d}. \end{aligned} \quad (15)$$

其中旧计划不同时安排充、放电。该估计作为新计划的起点；旧末槽实际执行时，放电还会受该槽实际需求限制，故 0:10 的真实储电量可能与估计值不同。执行器记录这一按需削减，并将真实值连续传递到次日，不重置为 6000 kWh。

6.6 滚动优化与终端库存策略

为利用最新预测信息并降低有限预测误差造成的调度偏差，微网能量管理中常采用滚动时域优化策略 [4, 5]。本文据此采用未来 7 天联合优化、仅执行当前日计划的滚动机制。

每天 0:00 以未来 $H = 7$ 天的因果预测协同优化，但只执行当天计划、次日滚动重算。视野内共 $H \times 144$ 个时段，记指标 $k = 0, \dots, H \times 144 - 1$ ；电价按日内曲线循环铺展， $p_k = p_{k \bmod 144}$ 。预测负荷与可用光伏 $\hat{L}_k, \hat{V}_k$ 是外生输入，由式 (9) 的七天前同刻值给出，不随决策变量变化。计划购电、光伏消纳、充放电和库存记为 $\hat{g}_k, \hat{v}_k, \hat{c}_k, \hat{d}_k, \hat{E}_k$ ，完整可行域为

$$\min \sum_{k=0}^{H \times 144 - 1} p_k \hat{g}_k, \quad (16)$$

$$\text{s.t. } \hat{g}_k + \hat{v}_k + \hat{d}_k = \hat{L}_k + \hat{c}_k, \quad (17)$$

$$\hat{E}_{k+1} = \hat{E}_k + \eta_c \hat{c}_k - \hat{d}_k / \eta_d, \quad (18)$$

$$\hat{q}_k^{\text{curt}} = \hat{V}_k - \hat{v}_k \geq 0, \quad 0 \leq \hat{v}_k \leq \hat{V}_k, \quad \hat{g}_k \geq 0, \quad (19)$$

$$0 \leq \hat{c}_k, \hat{d}_k \leq 5000 \Delta t = 833.333, \quad (20)$$

$$1200 \leq \hat{E}_k \leq 10800, \quad \hat{E}_0 = \hat{E}_0^{(d)} \text{ (式 (15))}, \quad \hat{E}_{H \times 144} = \hat{E}_0. \quad (21)$$

其中式 (15) 给出了由上一日实际库存与旧计划末槽动作推得的计划起点 $\hat{E}_0^{(d)}$ ；母线平衡、光伏消纳、购电与充放电约束对全部 $k = 0, \dots, H \times 144 - 1$ 成立，库存递推与库存边界含端点 $k = H \times 144$ 。

需要区分计划变量与执行变量：求解上式得到的是计划量 $(\hat{g}_k, \hat{c}_k, \hat{d}_k)$ ，不是真实执行量。当天真实负荷与光伏揭示后，只保留当天 144 槽的执行结果，并按式 (10)–(14) 把计划映射为执行量。记执行实际储能动作后尚待供给的母线需求为 $R_t = L_t + c_t^{\text{act}} - d_t^{\text{act}} \geq 0$ ，则

$$g_t^{\text{deliv}} = \min(\hat{g}_t, R_t), \quad g_t^{\text{unused}} = \hat{g}_t - g_t^{\text{deliv}} \geq 0. \quad (22)$$

实际光伏继续供给剩余需求，仍不足的部分形成紧急购电。储能动作以计划值为上界，仅在 SOC 上下界或实际需求不足时按 $\Delta c_t^{\text{sf}}, \Delta d_t^{\text{sf}}$ 削减；计划阶段已有的功率上限保持不变。该口径通过 7 天视野的库存回归约束限制有限视野末端行为，而实际库存始终按执行结果跨日传递。

6.7 模型求解

求解流程。按自然日顺序回放 2025 年 1 月 1 日至 12 月 31 日。每个自然日执行：先按跨午夜库存桥接规则，用 0:00 已知储电量与旧计划末槽动作估计 0:10 储电量并制定新计划；再执行旧计划末槽；最后从真实 0:10 储电量执行新一天的 144 槽计划。每天

144 槽的能量平衡、储电量转移、合同量、光伏恒等式与费用由独立核算器逐槽复算，任何一项不通过即判定不可行。1 月为预热，2 月 1 日至 12 月 31 日共 334 天计入正式评价。

全年正式结果。表 4 给出主口径（D-7 点预测 + 滚动 7 天视野）的全年费用分项。334 天全部可行，最大能量平衡残差在 10^{-9} 量级以内，跨日储电量连续 334 天无断裂。

表 4 问题二全年正式结果（2025-02-01 至 12-31，334 天）

指标	数值
全天总购电费（元）	16 886 077.67
正常合同购电费（元）	12 190 821.03
紧急购电费（元）	4 695 256.64
紧急购电量（kWh）	1 199 863.45
未使用合同电（kWh）	315 194.51
充电削减总量（kWh）	7 296.6
放电削减总量（kWh）	52 464.1
不可行天数	0
年末储电量（kWh）	8 824.35
日末储电量中位数（kWh）	8 664.6

预测方法的费用比较。预测误差指标各自强调不同性质，单一误差值不宜脱离实际决策目标解释 [7]。本文在同一滚动 7 天视野下比较两种因果预测（表 5）。虽然同星期日衰减平均在 1 月 15 日至 30 日的只读误差对比中负荷 MAE 更低（118.19 kW 对 142.05 kW），但其全年实际总费用为 17 295 362.55 元，比 D-7 的 16 886 077.67 元高约 409 285 元。该结果说明预测误差大小不能单独决定策略优劣，业务价值必须用统一核算器下的实际费用衡量。这里的费用是同一年度对候选方法的顺序回放比较，不是选模后的独立未来年份评价，因而只据此确定本文主报告方案，不外推为普遍最优方法。

表 5 两种因果预测的费用与误差对比：费用为 2025-02-01 至 12-31 的 334 天回放，MAE 为 01-15 至 01-30 的只读窗口（滚动 7 天视野）

预测方法	总费用（元）	紧急购电费（元）	紧急购电量（kWh）	负荷 MAE（kW）
七天前同刻 D-7	16 886 077.67	4 695 256.64	1 199 863.45	142.05
同星期日四周 衰减平均	17 295 362.55	5 137 792.14	1 300 168.64	118.19

联合分位扰动候选。分位数方法可用于表达不同的低估与高估代价 [8]。本文另测

一个仅使用历史残差的候选：负荷与光伏分别叠加同一 0.7/0.8/0.9 上分位残差。由于上调负荷与上调光伏对计划净负荷的作用方向相反，该构造并非缺电风险意义下的单向保守策略。在 2 月至 4 月、滚动 3 天的对照窗口内，点预测总费用最低（约 3 569 713 元）；三个联合分位候选分别约为 3 904 288、4 004 029、4 151 856 元，且正常购电费略低而紧急购电量更高（表 6）。因此本文只得出“这组联合边际扰动候选在该窗口表现较差”，不能据此否定净负荷分位、负荷上调且光伏下调等其他稳健设计；正式主方案仍采用点预测。

表 6 保守程度对费用的影响（2 月至 4 月窗口，同星期日衰减平均）

计划口径	总费用（元）	正常购电费（元）	紧急购电量（kWh）
点预测	3 569 713	3 305 336	71 203
0.7 分位	3 904 288	2 996 564	235 917
0.8 分位	4 004 029	2 998 400	259 266
0.9 分位	4 151 856	3 008 633	294 559

由此，问题二的主报告口径取 D-7 点预测与滚动 7 天视野。终端的处理不采用“每日首尾储电量相同”，因为在全年尺度上该口径会把储电量持续钉在 10800 kWh 上限，电池失去跨日调节能力，全年总费用升至 16 959 232 元；滚动 7 天视野的费用更低（16 886 078 元），且日末储电量中位数约 8665 kWh，既不贴上限也不贴下限。

6.8 结果分析与检验

费用结构。全年 16 886 077.67 元的购电费由 72.2% 的正常合同电费与 27.8% 的紧急购电费构成。334 天中每天均出现紧急购电，单日紧急购电量最高为 6 月 1 日的 22 831.6 kWh，全月紧急购电占比在 6 月达到 42.9%，在 2 月最低为 8.9%；夏季负荷与光伏波动更大，预测误差经 5 倍罚价放大，因此夏季紧急购电占比明显更高。未使用合同电全年 315 194.51 kWh，反映保守的合同量在光伏富余或负荷偏低时段的部分落空，其费用已计入正常合同电费。

储能轨迹。全年日末储电量中位数为 8664.6 kWh，最低 1818.7 kWh、最高 10 586.3 kWh，均未在任何一天结束时停留在 1200 或 10800 kWh 边界。日内储电量在光伏高峰可充至 10800 kWh、在负荷高峰可放至 1200 kWh，说明电池在物理边界内得到充分利用而未长期贴边。全年充电削减 7296.6 kWh、放电削减 52 464.1 kWh，均来自实际负荷与预测偏差触发的物理边界保护，规模相对全年吞吐量很小。

独立验证。针对执行与核算链，构造两项独立验收。其一，三天连续窗口（1 月预热 + 2 月 1 日至 3 日）在两种预测下的全部 144 槽记录经独立核算器复算，覆盖槽位与时间连续性、储电量与功率边界、光伏与合同量恒等式、费用复算、逐日与跨日储电量

连续，以及每个决策时点的输入因果性，均无违反。其二，紧急购电时段合并规则以构造记录单测，重点覆盖跨午夜边界。此外，问题二的改动未破坏问题一已验证的内核：主探针与独立交叉验证复跑保持一致。

综上，问题二在严格因果的信息约束下，以 D-7 点预测、滚动 7 天视野与独立核算器建立了可复现的全年购电策略，334 天全部可行，总购电费 16886077.67 元。完整计划购电量、充放电量与紧急购电结果按题面要求写入 `result2.xlsx`。

七、问题三的模型的建立和求解

7.1 具体分析

问题三的新增信息不是当天光伏真值，而是日内逐次发布的未来 24 小时光伏预报。因此，本问的核心不是重新建立一套储能模型，而是在问题二的物理执行器上加入“预测发布-计划改版-按版结算”的闭环。预报更新可以减少预测偏差引起的紧急购电，但频繁改版又会产生上调溢价和下调冲回，故是否需要 6:00、12:00、18:00 预报必须由实际总费用、紧急购电量和调整规模共同判断，不能只比较预测误差。

7.2 预测数据展开与时间对齐

记日内发布时点为 $r \in \{0, 6, 12, 18\}$ ，其对应当日的起始槽位为 s_r ，即 $s_0 = 0$ 、 $s_6 = 35$ 、 $s_{12} = 71$ 、 $s_{18} = 107$ 。0:00 先形成初始计划，6:00、12:00 和 18:00 分别加入当时刚发布的附件 3 光伏预报；任一时刻只使用发布时间不晚于该时刻的数据。按照时间方案 A，三个调整边界分别对应槽位 35、71 和 107，故新版本从 [6:00, 6:10)、[12:00, 12:10) 和 [18:00, 18:10) 起生效。附件 3 第 h 列保留为 $\text{valid_time} = \text{issue_time} + h$ 的点预测，不把首小时左移。

对相邻整点预报作分段线性插值；发布时刻的左端锚点只取此前已经发布的批次，2025 年 1 月 1 日 0:00 无历史锚点的首小时回退附件 1 先验。用不进入正式评价期的一月选择展开方法：4429 个可比较槽上，线性展开的 MAE、RMSE 分别为 61.93 kW、133.09 kW，前值保持分别为 242.04 kW、463.22 kW，故采用线性展开。当前批次跨午夜仍有效的 24 小时预测完整保留，超过覆盖范围后才回退严格因果的 D-7 光伏基线；负荷预测仍沿用问题二的 D-7 方法。

7.3 滚动重调度机制

模型预测控制通过不断获取新的系统信息、重新求解有限时域优化问题并仅实施近期决策，实现对预测误差的动态修正 [1, 4, 5]。本文将这一思想用于日内光伏预报更新，并保持结算版本可追溯。

每次求解均保留问题二的 7 天视野和视野末库存回归约束，只执行到下一发布时点；已经执行的槽位永久锁定，只允许重优化尚未执行的时段。记时点 r 的预测视野为 $\mathcal{H}_r$ ，其中当日之后的后续日期记为 $\mathcal{H}_r^{\text{future}}$ 。当前批次对未来 24 小时提供光伏信息，更远时段用 D-7 补齐；这种处理既利用新信息，又避免短视地在更新边界耗空电池。0:00 跨日衔接仍沿用问题二中确立的跨日库存桥接规则：先依据旧计划末槽动作估计新计划起始库存，再执行旧日最后一槽，实际库存连续传给下一日。

7.4 调整结算与成本核算

设 $q_t^{(r)}$ 为发布时点 r 确定的时段 t 合同购电量， $q_t^{(r-)}$ 为调整前最近一版合同量， $a_t^{r,+}$ 、 $a_t^{r,-}$ 为相对上一版的上调量与下调量。只对 $t \geq s_r$ 的未执行时段定义非负调整量：

$$q_t^{(r)} = q_t^{(r-)} + a_t^{r,+} - a_t^{r,-}, \quad a_t^{r,+}, a_t^{r,-} \geq 0. \quad (23)$$

0:00 初始计划沿用问题一的供需平衡、光伏、功率和 SOC 约束，在 7 天预测视野内最小化普通购电费。日内更新时，对当日已经签约但尚未执行的部分，目标函数改为相邻版本的调整现金流；对视野中尚未签约的后续日期仍按普通购电费计价：

$$\min \left\{ \sum_{t=s_r}^{143} (1.5p_t a_t^{r,+} - 0.5p_t a_t^{r,-}) + \sum_{t \in \mathcal{H}_r^{\text{future}}} p_t g_t \right\}. \quad (24)$$

两部分共享同一 SOC 状态链和视野末约束。求得主目标最优值后，再以充放电与调整总量最小作词典序第二目标，消除同价时段无意义的往返动作。

实际执行时，合同电、电池和光伏仍遵循问题二的统一执行规则；预测不足形成的缺口由 5 倍电价紧急购电补齐。记问题三按版本账本核算的全年总费用为 C_3 ，其表达式为

$$C_3 = \sum_t p_t q_t^{(0)} + \sum_{r \in \{6,12,18\}} \sum_{t \geq s_r} (1.5p_t a_t^{r,+} - 0.5p_t a_t^{r,-}) + \sum_t 5p_t g_t^{\text{emg}}. \quad (25)$$

其中下调项是按题面下调规则对被替换合同量的 0.5 倍冲回，不是额外罚款；最终合同量 q_t^{final} 不再重复收取普通购电费。

7.5 模型求解

采用 HiGHS 对每个发布时点的稀疏线性规划顺序求解。2025 年 1 月用于预测与库存预热，正式评价期为 2 月 1 日至 12 月 31 日共 334 天。四时点主方案的费用与电量分解见表 7。

题面指定的四个日期汇总见表 8。0:00 和 24:00 储电量、六个四小时分块均由逐槽记录的真实时间戳重新聚合，而不是按槽位位置切块；详细分块见附录表 12，全部 10 分钟计划及紧急购电区间保存在正式结果文件中。

表 7 问题三四时点滚动策略的全年结果

类别	数值	单位
初始合同购电费	12317106.21	元
上调购电费	600179.55	元
下调冲回	244787.55	元
净调整现金流	355392.01	元
紧急购电费	3133253.56	元
总购电费	15805751.77	元
上调量/下调量	618742.57/754150.54	kWh
紧急购电量/未用合同电量	809227.14/304721.36	kWh

表 8 问题二、三四个指定日期结果摘要

问题	日期	当日费用/元	紧急购电/kWh	0:00 SOC/kWh	24:00 SOC/kWh
二	3 月 20 日	45671.26	1091.82	8769.36	4839.38
二	6 月 21 日	28014.65	1424.94	2719.43	8993.57
二	9 月 23 日	61537.57	4913.93	8816.15	8845.57
二	12 月 21 日	67079.69	2171.33	8933.95	8839.97
三	3 月 20 日	52681.32	3338.74	8769.36	4839.38
三	6 月 21 日	25232.54	850.62	2708.41	8842.26
三	9 月 23 日	49871.39	1778.48	8666.15	8695.57
三	12 月 21 日	68230.84	2383.64	8783.96	8689.97

问题二总费用为 16 886 077.67 元；将 0:00 光伏输入换为附件 3 预报并按问题三口径制定初始计划后，费用降至 16 350 694.80 元，减少 535 382.87 元；再加入 6:00、12:00、18:00 更新，费用降至 15 805 751.77 元，继续减少 544 943.03 元。两段合计减少 1 080 325.89 元、降幅 6.3977%。因此，只有第二段能够归因于追加日内更新；第一段同时包含预报来源和初始计划变化。最终紧急购电量较问题二减少 390 636.31 kWh，334 天均可行，SOC 始终位于 1200–10800 kWh。

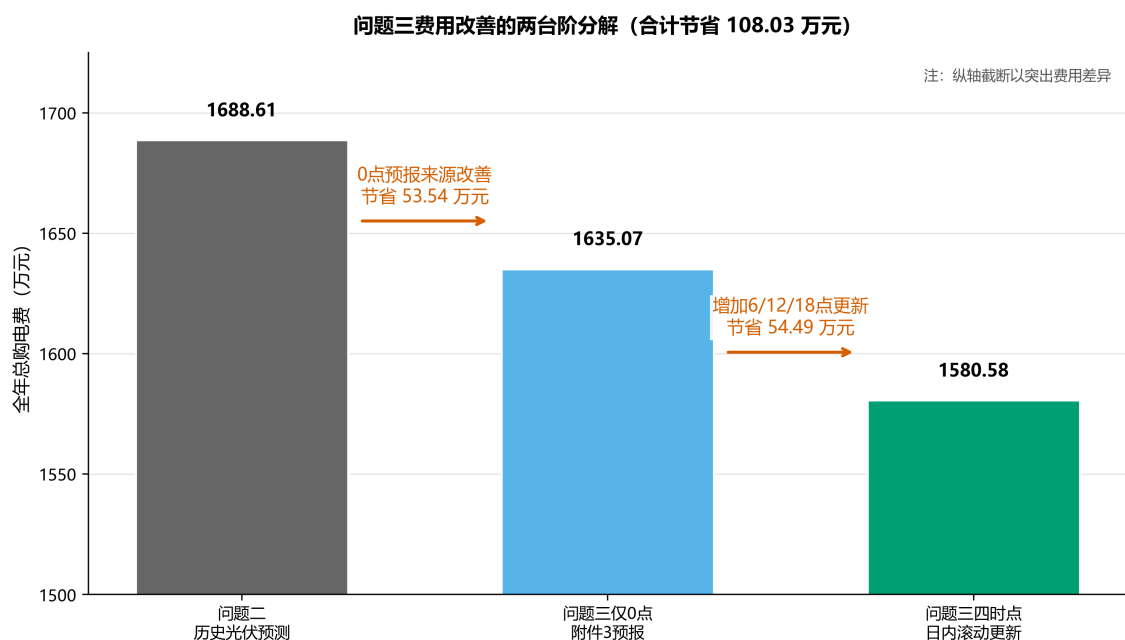


图 1 问题三相对问题二的两阶段费用改善

更新时点消融。保持数据、执行器、结算器和预测展开完全一致，依次加入 6:00、12:00 和 18:00 批次，结果见表 9。6:00 使总费用下降 341695.91 元、紧急购电量下降 159601.56 kWh；12:00 在此基础上再降低 203250.02 元和 70515.50 kWh。18:00 相较截至 12:00 的方案反而轻微增加 2.90 元和 0.27 kWh，费用变化低于 0.001%、电量变化低于百万分之一。因此，6:00 与 12:00 预报具有明确经济价值，18:00 在当前数据上不具实质边际收益；没有证据支持再引入更密集的其他发布时间。由于附件未提供其他发布时间的预报，本文不对其作无法验证的收益推断。

表 9 日内光伏预报更新时点的逐级消融

可用发布时点	总费用/元	净调整费/元	紧急购电量/kWh	不可行天数
00	16350694.80	0.00	1039343.92	0
00+06	16008998.90	271160.45	879742.36	0
00+06+12	15805748.88	355391.71	809226.86	0
00+06+12+18	15805751.77	355392.01	809227.14	0

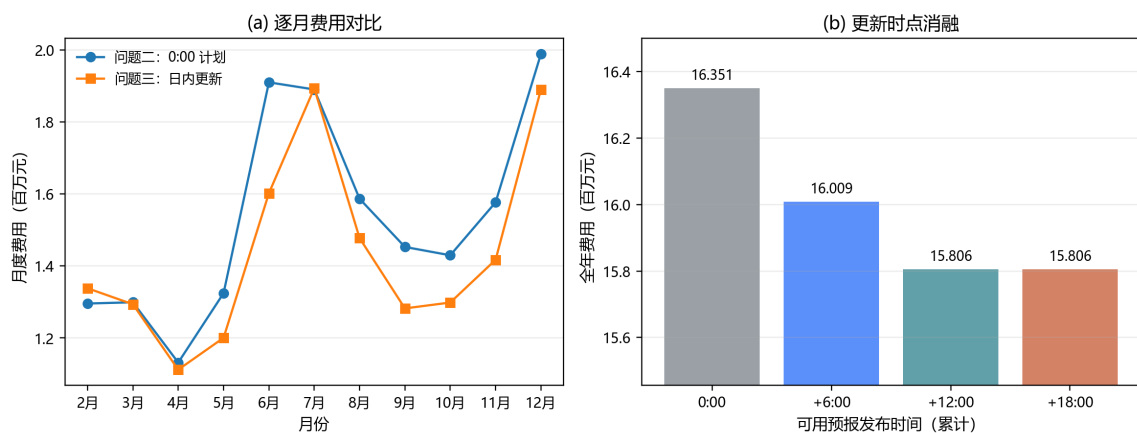


图2 问题二、问题三月度费用及更新时点消融

图3以四个题定日期中问题二至问题三费用改善最大的9月23日为例，展示预报发布、合同调整、紧急购电和SOC的联动。6:00与12:00之后合同曲线随新信息调整，储能在光伏富余时充电、晚高峰放电；橙色紧急购电面积对应预测偏差未被合同与储能覆盖的缺口。

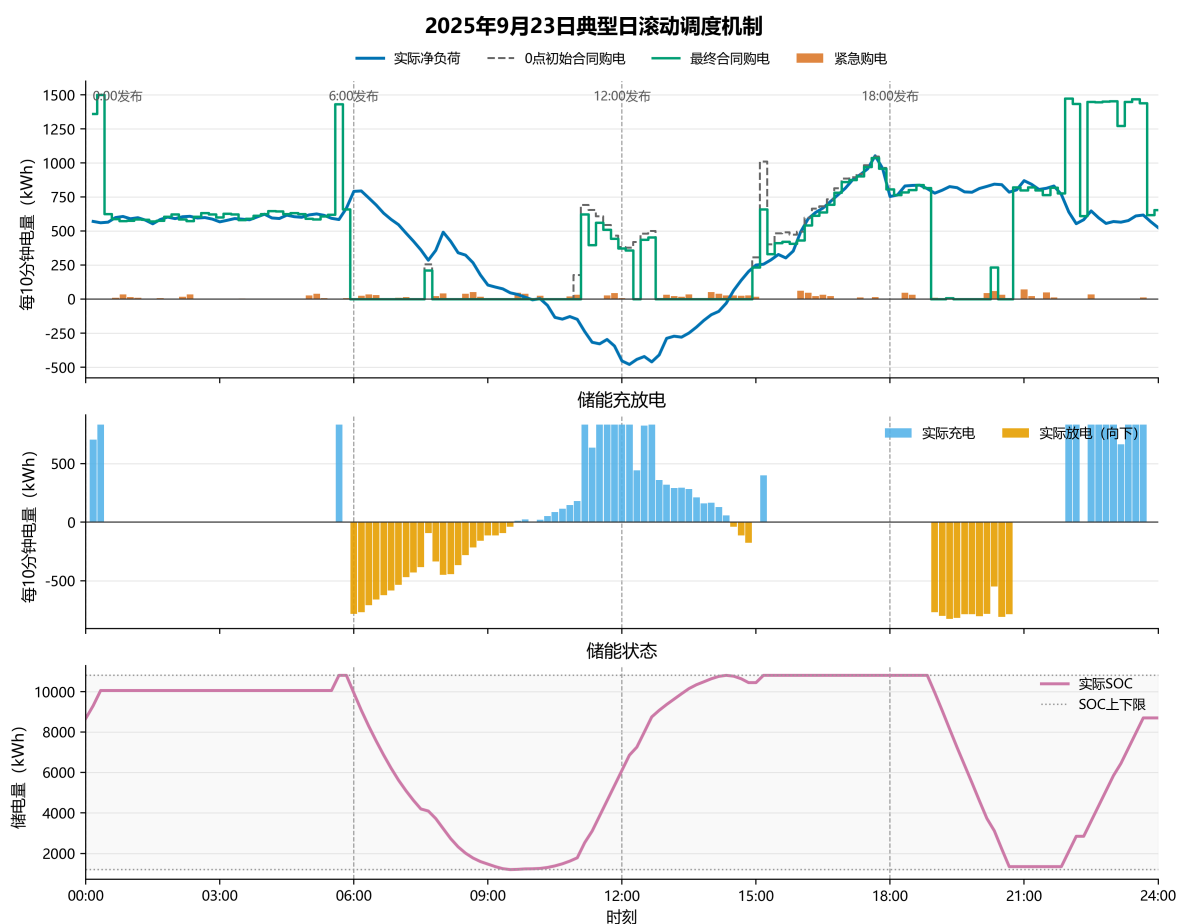


图3 2025年9月23日问题三滚动调度机制

独立验证与结果文件。独立脚本不调用优化器，直接由逐槽记录和计划版本复算。母线平衡和 SOC 递推最大残差分别为 7.96×10^{-13} kWh 和 5.46×10^{-12} kWh，跨日 SOC 与时间轴均连续；版本顺序、目标槽定价、逐日费用及工作簿回读一致。最终策略写入 result3.xlsm。

上述结论是在固定日内电价、D-7 负荷预测、线性光伏展开和相邻合同版本现金流结算口径下得到的严格因果顺序回放结果。每次 LP 在给定信息集下最优，但预测过程无未来泄漏不等于方法选择层具有独立样本外评价，故不把它表述为未来年份或不确定环境下的全年全局最优。

八、问题四的模型的建立和求解

8.1 具体分析

问题四不改变储能的物理结构，也不新增可控设备；变化在于外网电价由固定日内曲线改为逐日波动曲线。因此本问的数学任务是带价格不确定性的滚动优化与事后策略评价。若直接把附件 4 的未来真实价格送入优化器，虽然费用更低，却等于在 0:00 提前知道当天乃至后续数日价格，会造成信息泄漏。本文采用双价格通道：预测价只负责制定计划，真实价只负责执行后的费用结算，并另设完美信息模型作为不可执行下界。

8.2 信息集约束与评价机制

本问严格分离“决策时可得的预测价”与“事后结算的真实价”。记 p_t 为附件 4 中目标槽的真实电价， $\hat{p}_{t|r}$ 为发布时刻 r 能得到的预测价。优化器只使用决策时刻 r 及之前已观测的历史电价形成预测；附件 4 的目标槽真实价只用于执行回放与费用核算，不进入任何一次计划优化，否则等同于提前获知未来价格、造成信息泄漏。所有滚动视野共享同一真实决策时刻。

预测模型应在严格隔离未来信息的样本外框架下评价，并与简单基准方法比较 [9]。因此，本文只用不进入正式评价期的一月进行价格预测方法的顺序比较，并在正式评价期开始前确定预测器。其中 1 月 1 日至 14 日为预热（最早的四周同槽历史尚未完整观测），实际比较窗口为 1 月 15 日至 30 日；2 月 1 日至 12 月 31 日的 334 天完全不参与价格方法选择。七天前同刻法与同星期日衰减加权法在该窗口两条分支上的合计实际费用分别为 2 069 387.34 元和 2 065 548.54 元，故选用后者。对目标槽 t ，取此前第 7, 14, 21, 28 天同槽且已完成观测的价格，权重依次为 1, 0.8, 0.8², 0.8³：

$$\hat{p}_{t|r} = \frac{\sum_{j \in \mathcal{A}_{t,r}} 0.8^{j-1} p_{t-7j}}{\sum_{j \in \mathcal{A}_{t,r}} 0.8^{j-1}}, \quad \mathcal{A}_{t,r} = \{j : p_{t-7j} \text{ 在 } r \text{ 时已观测}\}. \quad (26)$$

若集合为空则回退附件 1 典型日价格，来源观测时间均不晚于决策时刻。正式期第二问式价格预测的 MAE 和 RMSE 分别为 0.04672 元/kWh 和 0.06649 元/kWh。评价机制上，

另设使用未来真实负荷、光伏与电价的一次性完美信息 LP 作为不可执行下界，只用于量化负荷、光伏与电价联合信息下不可再降的费用下界，不参与正式计划，也不用于填写结果模板。

8.3 模型建立

第二问式分支完全复用问题二的 D-7 负荷与光伏点预测、7 天滚动视野、跨日库存桥接和执行安全削减，只将计划目标中的固定价格替换为 $\hat{p}_{t|0}$ ：

$$\min \sum_{t \in \mathcal{H}_0} \hat{p}_{t|0} g_t, \quad (27)$$

约束仍为母线能量平衡、光伏消纳、充放电功率、SOC 上下界及视野末库存回归。记第二问式分支的全年实际费用为 C_{4-2} ，由真实目标槽价格核算：

$$C_{4-2} = \sum_t p_t q_t^{(0)} + \sum_t 5 p_t g_t^{\text{emg}}. \quad (28)$$

第三问式分支复用问题三的 0:00、6:00、12:00、18:00 四时点更新。负荷仍用 D-7 预测，光伏使用当时已发布的附件 3 点预测线性展开，价格使用 $\hat{p}_{t|r}$ 。第 r 次更新的优化目标为

$$\min \left\{ \sum_{t=s_r}^{143} (1.5 \hat{p}_{t|r} a_t^{r,+} - 0.5 \hat{p}_{t|r} a_t^{r,-}) + \sum_{t \in \mathcal{H}_r^{\text{future}}} \hat{p}_{t|r} g_t \right\}. \quad (29)$$

结算时将上式中的预测价全部换为真实目标槽价格。0:00 初始计划费、逐次调整现金流与紧急购电费构成唯一总账，最终合同量不再重复计费。

8.4 模型求解

两条分支均从 2025 年 1 月 1 日顺序预热库存，2 月 1 日至 12 月 31 日共 334 天进入正式评价。价格方法确定后才运行正式窗口。表 10 给出独立核算器复算的费用与电量。

表 10 波动电价下两条正式策略的全年结果

指标	第二问式	第三问式
初始/正常合同购电费（元）	12 814 582.52	12 964 681.98
调整净现金流（元）	—	381 722.38
紧急购电费（元）	4 928 742.36	3 259 691.08
总费用（元）	17 743 324.88	16 606 095.44
紧急购电量（kWh）	1 202 608.49	809 107.17
未使用合同电（kWh）	315 104.41	305 069.59
上调量/下调量（kWh）	—	617 426.96/758 273.72
不可行天数	0	0

第三问式比第二问式节省 1 137 229.44 元，降幅 6.4093%；紧急购电量减少 393 501.32 kWh，降幅 32.7207%。这一差异是更换光伏预报来源并加入日内更新后的合并效果：紧急购电费明显下降，其中一部分被 381 722.38 元的上调净现金流抵消，其余形成净节省。现有对照不能把全部差额单独归因于日内更新。图 4 显示各月均按真实波动价格结算，而非用预测价格替代账单。

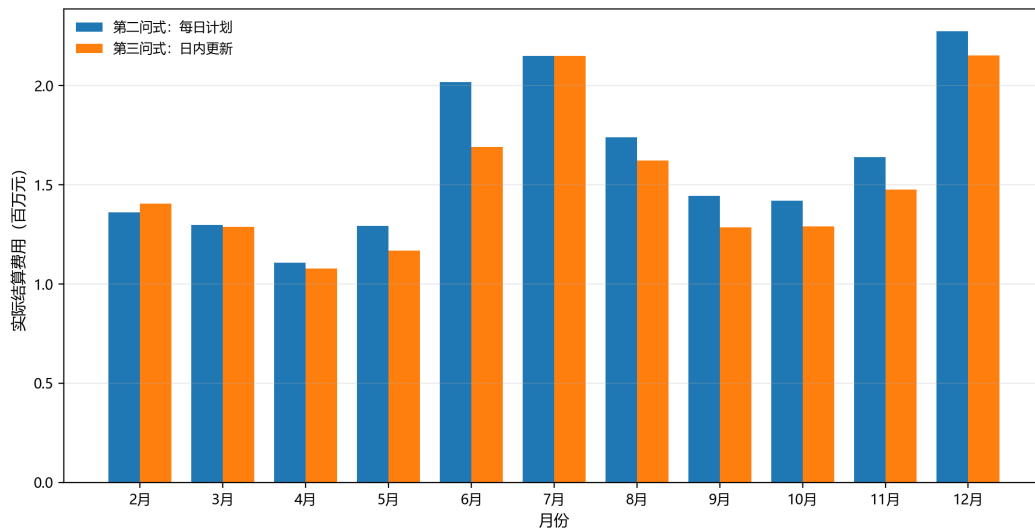


图 4 波动电价下两条策略的逐月实际结算费用

为衡量信息不足造成的代价，另以全年真实负荷、光伏和电价一次性求解完美信息 LP，并分别固定为对应正式回放的期初与期末 SOC。第二问式和第三问式的下界分别为 12 783 515.84 元和 12 783 514.18 元，因果策略高出 38.7985% 和 29.9024%。该结果只说明预测改进仍有空间，不是可执行策略，也不用于填写结果模板。

独立验证器不调用生产回放器，直接读取附件 4、逐槽记录、版本日志和结果工作簿。两条分支均覆盖 334 天、每天 144 槽；真实电价逐值误差为 0，能量平衡最大残差不超过 6.83×10^{-13} kWh，SOC 递推最大残差不超过 5.46×10^{-12} kWh，跨日 SOC 无断裂；目标槽定价、版本顺序和防重复收费也得到复算。最终结果写入 result4-2.xlsm 与 result4-3.xlsm。

九、模型的分析与检验

9.1 竞赛化验证体系

本文不以测试数量代替模型可信度，而按数据口径、物理可行、经济核算、决策增益和提交一致五层验证。前两层回答“数据和结果是否算对”，第三层排除费用重算或漏算，第四层判断新增机制是否真正改善实际成本，第五层保证论文结论与提交文件一致。本文选择能覆盖本题主要风险的最小验证组合，见表 11。

表 11 面向建模比赛的精简验证体系

层次	验证对象	本文方法	通过标准
数据口径	时间、单位、附件	144 槽及官方数据逐值检查	无缺失、重复和未来信息
物理可行	平衡、SOC、功率	从正式 CSV 逐槽独立复算	残差低于容差且零越界
经济核算	合同、调整、紧急费	独立费用账本	分项之和与总费用闭合
决策增益	储能与更新信息	无储能基线、时点消融、理想下界	同一信息集和核算器下比较
提交一致	论文与五份结果	工作簿回读和指定日期核对	数值、时钟和汇总一致

问题一另用显式充放电互斥 MILP 与主 LP 对拍，以排除连续模型依赖无意义的同时充放电。完整 KKT 条件多由同一求解器的对偶量计算，不能发现漏约束、错时钟或错导出；动态规划又需离散 SOC 并引入网格误差，故二者不再重复堆入正文。工作簿逐单元格回读、文件散列和合成反例保留在支撑材料中。

9.2 灵敏度分析

初始储电量和最大充放电功率变化对问题一费用的影响均小于 0.35%。效率更敏感：充、放电效率由 0.9 降至 0.855 时费用增加 3.7606%，升至 0.945 时费用降低 3.4978%；

若将题目“90%”解释为往返效率并取 $\eta_c = \eta_d = \sqrt{0.9}$ ，费用比主口径低 3.7733%。因此本文重点保留效率口径敏感性，其余参数只作为结论稳定性的辅助说明。

对负荷和光伏施加有界时间相关扰动后，模型仍可行，相对无储能基线的节省率保持在 23.97%–29.80%。该检验只说明问题一结论对局部输入变化不敏感，不外推为问题二至四预测策略的全年稳健性。

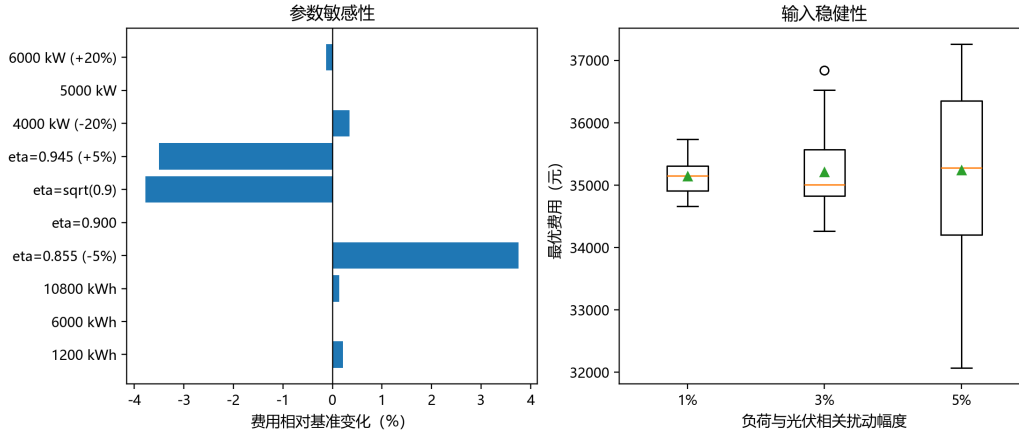


图 5 参数敏感性与输入稳健性

9.3 独立复算与误差分析

独立验证代码不调用生产 LP、执行器和核算器，而是直接从附件 1 与最终 CSV 逐槽复算。144 槽无缺失、重复或非有限值；负荷和光伏的单位换算最大误差分别为 1.14×10^{-13} kWh 和 2.27×10^{-13} kWh；能量平衡最大残差为 2.27×10^{-13} kWh，储电量转移最大误差为 1.82×10^{-12} kWh。储电量、功率和购电语义违反次数均为 0，CSV、论文汇总表和正式工作簿的购电量最大差为 6.82×10^{-13} kWh。

进一步建立显式保留 145 个储电量状态、用二进制变量强制充放电互斥的 MILP。其费用为 35126.948589 元，与主 LP 的差异处于浮点误差量级；两个模型均无同时充放电。逐槽动作因同价时段的多重最优而可能不同，但总费用、全天购电量和终端库存一致。因此主结果同时得到物理账本、简单无储能基线和替代优化模型支持。

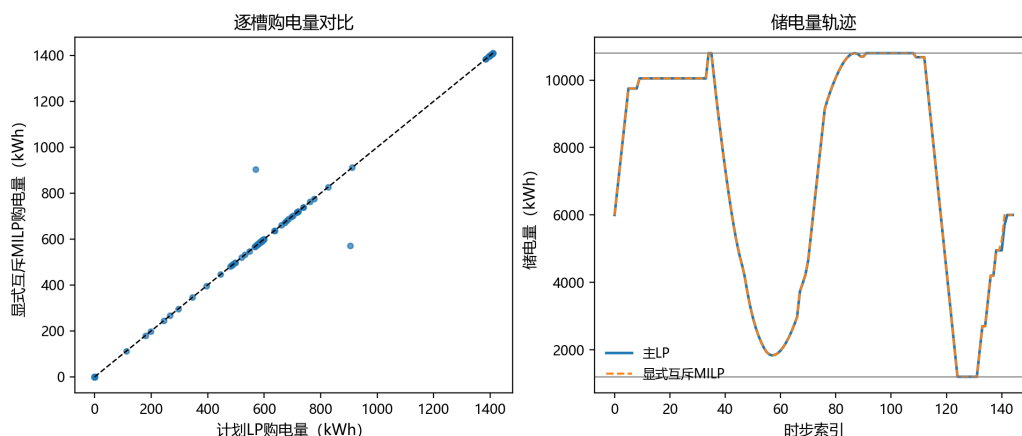


图 6 主模型与替代模型验证

问题一是确定性优化，不包含拟合或预测参数，因此传统随机 K 折验证并不适用。输入扰动仅作为局部压力测试，不能替代问题二至问题四的全年顺序回测。

十、模型的评价、改进与推广

10.1 模型的优点

- 四问共用母线能量平衡、SOC 递推和功率边界，新增问题只改变信息集与结算层，避免重复模型产生口径冲突。
- 预测、计划、执行和核算相互分离；每条历史与预报均记录可观测时刻，能够直接审计是否提前读取未来数据。
- 正式结果均由独立核算器从逐槽记录复算，并对官方工作簿回读，费用结论可追溯到具体槽位和计划版本。

10.2 模型的缺点

- 电价预测仅利用同星期历史槽位，虽然实现稳定且易审计，但没有显式利用节假日、天气或负荷与电价的联合变化。
- 模型未显式计入电池循环老化成本，而充放电深度与循环频率会影响电池寿命及真实运行经济性 [10]。题目没有给出电池类型、循环寿命曲线、购置或更换成本和温度条件，若直接套用外部磨损单价会把无法校准的假设带入正式购电目标，因此本文只将其列为局限。

10.3 高级因素的取舍

高级方法只有在同一信息集、执行器和费用账本下稳定改善结果，才有引入价值。样本平均近似（SAA）和条件风险价值（CVaR）适合研究预测误差与尾部成本，但本题

只有一年高维轨迹，且没有给定风险偏好；若没有仅用历史残差生成场景、固定风险参数并留出评价期，复杂模型容易产生样本内乐观。故本文不把 SAA/CVaR 升格为正式策略，只将其作为获得更长历史数据后的同口径对照方向。

场景树是多阶段随机规划的实现结构，不是独立创新点。问题二每天只有一次计划，问题三虽然有四个发布时点，但附件已经给出实际到达的预报版本；当前在 0:00、6:00、12:00、18:00 锁定已执行时段并重算未来计划，本质上已经是题目时间粒度下的确定性 MPC。另加场景树需要估计未来预报版本的条件分布，另加十分钟“实时闭环”又会改变储能动作是否允许即时修改的执行权限，均缺少题面数据和规则支撑，故不进入正式结果。

10.4 模型的改进

在获得多年预测版本、实际值及电池参数后，可先用仅来自决策时点以前的联合残差生成少量场景，再以滚动留出数据比较确定性 MPC、SAA 和均值-CVaR 策略，同时报告实际总费用、紧急购电量、尾部日成本与场景稳定性。电池寿命可先用吞吐成本区间做敏感性，再在参数充分时加入循环深度退化模型；无论采用何种扩展，都保留本文确定性 LP 作为可解释基线和核算检查。

10.5 模型的推广

该框架适用于含储能和间歇式新能源的园区、楼宇或小型微网。只要替换设备参数、价格规则和预测接口，即可在不改变执行与核算链的情况下生成滚动购电计划；若允许反送电或参与辅助服务，则需新增售电变量与相应结算约束。

AI 工具使用声明

本文在题意梳理、建模方案讨论、程序编写与调试、结果核查、文档整理和语言润色过程中使用了 AI 工具，包括 OpenAI Codex，以及 Claude Code 与 DeepSeek 组合（内置模型为 DeepSeek V4.1 Flash）。所有模型假设、决策口径、程序改动、实验结果和论文内容均由参赛队员逐项复核，核心数值由独立核算程序重新计算。详细使用过程随支撑材料提交于《AI 工具使用详情.pdf》。

参考文献

- [1] OLIVARES D E, MEHRIZI-SANI A, ETEMADI A H, et al. Trends in microgrid control [J/OL]. IEEE Transactions on Smart Grid, 2014, 5(4):1905-1919. DOI: 10.1109/TSG.2013.2295514.

- [2] BOYD S, VANDENBERGHE L. Convex optimization[M]. Cambridge: Cambridge University Press, 2004.
- [3] 司守奎, 孙玺菁. 数学建模算法与应用[M]. 北京: 国防工业出版社, 2011.
- [4] RAWLINGS J B, MAYNE D Q, DIEHL M M. Model predictive control: Theory, computation, and design[M]. 2nd ed. Madison: Nob Hill Publishing, 2017.
- [5] PARISIO A, RIKOS E, GLIELMO L. A model predictive control approach to microgrid operation optimization[J/OL]. IEEE Transactions on Control Systems Technology, 2014, 22(5):1813-1827. DOI: 10.1109/TCST.2013.2295737.
- [6] VIRTANEN P, GOMMERS R, OLIPHANT T E, et al. Scipy 1.0: Fundamental algorithms for scientific computing in python[J/OL]. Nature Methods, 2020, 17:261-272. DOI: 10.1038/s41592-019-0686-2.
- [7] HYNDMAN R J, KOEHLER A B. Another look at measures of forecast accuracy[J/OL]. International Journal of Forecasting, 2006, 22(4):679-688. DOI: 10.1016/j.ijforecast.2006.03.001.
- [8] KOENKER R, BASSETT G. Regression quantiles[J/OL]. Econometrica, 1978, 46(1): 33-50. DOI: 10.2307/1913643.
- [9] TASHMAN L J. Out-of-sample tests of forecasting accuracy: An analysis and review[J/OL]. International Journal of Forecasting, 2000, 16(4):437-450. DOI: 10.1016/S0169-2070(00)00065-0.
- [10] XU B, ZHAO J, ZHENG T, et al. Factoring the cycle aging cost of batteries participating in electricity markets[J/OL]. IEEE Transactions on Power Systems, 2018, 33(2):2248-2259. DOI: 10.1109/TPWRS.2017.2733339.

附录 A 文件列表

文件名	功能描述
slot_adapter.py	144 槽位适配器
wallclock_output.py	真实墙钟时间戳聚合层
lp_kernel.py	确定性 LP 内核与双引擎求解
executor.py	执行器与计划版本账本
accountant.py	独立核算器
template_io.py	官方模板原位填写与回读
figure_style.py	论文图中文字体与字号统一配置
data_io.py	附件读取与防泄漏接口
validate_official_attachment2.py	官方附件 2 光伏/负荷读取核对
extract_required_dates.py	四个指定日期摘要抽取
generate_q1_result.py	Q1 正式结果生成与审计
validate_q1.py	Q1 结果独立验证
q2_forecast.py	Q2 因果预测候选与冷启动
q2_conservative.py	Q2 分位保守计划候选
q2_replay.py	Q2 顺序回放与终端库存口径
generate_q2_result.py	Q2 正式 result2 生成、回读与审计
q2_three_day_acceptance.py	Q2 三天连续窗口独立验收
q2_emergency_merge_tests.py	Q2 紧急购电时段合并测试
q3_forecast.py	附件 3 点预测的因果 10 分钟展开
q3_optimizer.py	Q3 初始计划与日内调整的稀疏 LP
q3_replay.py	四时点顺序回放与合同版本记录
generate_q3_result.py	Q3 全年正式结果生成
finalize_q3_result.py	Q3 官方结果模板生成
plot_q3_evidence_figures.py	Q3 收益分解与典型日机制图生成
validate_q3_result.py	Q3 物理、账本与工作簿独立验证
q4_price.py	Q4 严格因果电价预测
q4_energy.py	Q4 复用 Q2/Q3 能量预测的因果适配器
q4_replay.py	Q4 双价格滚动回放与版本结算
generate_q4_results.py	Q4 两条全年正式回放
finalize_q4_results.py	Q4 官方结果模板生成
validate_q4_results.py	Q4 物理、价格、账本与工作簿独立验证

附录 B 核心求解代码

下面按模块摘录四问共用的主干求解代码，完整可运行源程序（含全部生成、验证与绘图脚本）请见支撑材料源码/目录。

2.1 时间槽适配与统一线性规划内核

```
1  """144 槽位适配器：输入数据列 → slot_id → 结果模板列。
2
3  冻结口径（DECISIONS.md D002 P0-1 + D003，方案A，起点对齐、不平移）：
4  - 官方输入表 144 列标签为 '00:10'...'23:50', '0:00+1';
5    官方结果模板 144 行区间为 '0:10-0:20'...'23:50-0:00+1', '0:00+1-0:10+1'。
6  - 输入列 k、槽位 k、模板行 k 按位置一一对应（无平移）：
7    槽位 k 的区间 = [标签_k, 标签_k + 10min):
8      slot 0    = [00:10, 00:20)
9      slot 142 = [23:50, 24:00) ← 跨日区间（终点 = 次日 0:00）
10     slot 143 = [24:00, 24:10) ← 次日第一个 10 分钟（标签 0:00+1）
11  - 计算以整数 slot_id = 0..143 为唯一主键，禁止用时间字符串拼接表。
12  - 映射集中在本模块；若未来推翻方案A，必须同时修改终端SOC、模板解释和测试。
13
14  注意（D002 冻结口径的推论，已在探针报告记录）：
15  该映射下一天的 144 槽覆盖 [00:10, 次日 00:10)；当天 [00:00, 00:10)
16  没有输入列（官方表无 00:00 列），不参与调度（无动作，SOC 不变）。
17  Q1 的 0:00 储电量 = E_0（附录初值），24:00 储电量 = E_143
18    （第 143 个槽开始时刻），E_144 为次日 00:10 的 SOC。
19  D002 P0-2 冻结的终端约束 E_144 = E_0 视为"144 槽周期首尾相等"。
20  """
21
22  from __future__ import annotations
23
24  from dataclasses import dataclass
25  from datetime import date, datetime, timedelta
26
27  N_SLOTS = 144
28  SLOT_MINUTES = 10
29
30
31  @dataclass(frozen=True)
32  class Slot:
33      """一个 10 分钟槽位（冻结的起点对齐口径）。"""
34
35      slot_id: int          # 0..143
36      day: date             # 提供输入值的自然日（槽 143 的区间落在次日）
37      interval_start: datetime
38      interval_end: datetime
39      source_label: str     # 输入表点标签：00:10...23:50 或 "0:00+1"
40      template_interval: str # 模板区间标签："0:10-0:20" ... "0:00+1-0:10+1"
41
42      @property
43      def source_column(self) -> int:
44          """输入宽表中的列号（第 k+2 列：第 1 列是日期）。"""
45          return self.slot_id + 2
46
47      @property
```

```

48     def template_column(self) -> int:
49         """结果模板中的列号（第 k+2 列：第 1 列是日期）。"""
50         return self.slot_id + 2
51
52     @property
53     def is_cross_day(self) -> bool:
54         """区间是否跨自然日边界（slot 142: 23:50→24:00）。"""
55         return self.interval_start.date() != self.interval_end.date()
56
57     @property
58     def is_next_day_slot(self) -> bool:
59         """区间是否完全落在次日（slot 143: 0:00+1-0:10+1）。"""
60         return self.interval_start.date() != self.day
61
62
63     def _fmt_t(dt: datetime) -> str:
64         # 官方模板/输入表风格：小时不补零、分钟两位（'0:10'、'23:50'）
65         return f"{dt.hour}:{dt.minute:02d}"
66
67
68     def _fmt_source_label(start: datetime) -> str:
69         """输入表点标签：次日 0:00 写作 '0:00+1'（官方写法）。"""
70         if start.hour == 0 and start.minute == 0:
71             return "0:00+1"
72         return _fmt_t(start)
73
74
75     def _fmt_template_interval(day: date, start: datetime, end: datetime) -> str:
76         """模板区间标签（官方写法）。"""
77         s = _fmt_t(start)
78         e = _fmt_t(end)
79         if start.date() == day and end.date() == day + timedelta(days=1):
80             # 当天 23:50 → 24:00（次日 0:00）
81             return f"{s}-0:00+1"
82         if start.date() == end.date() == day + timedelta(days=1):
83             # 次日 00:00 → 00:10
84             return f"{s}+1-{e}+1"
85         return f"{s}-{e}"
86
87
88     def build_day_slots(day: date) -> list[Slot]:
89         """构造某自然日全部 144 个槽位（冻结口径）。
90
91         slot k 的区间 = [day 00:00 + 10(k+1) 分钟, +10 分钟):
92         k=0 → [00:10,00:20), k=142 → [23:50,24:00), k=143 → [24:00,24:10)。
93         """
94         slots = []
95         base = datetime.combine(day, datetime.min.time())
96         for k in range(N_SLOTS):
97             start = base + timedelta(minutes=SLOT_MINUTES * (k + 1))
98             end = start + timedelta(minutes=SLOT_MINUTES)
99             slots.append(
100                 Slot(
101                     slot_id=k,
102                     day=day,
103                     interval_start=start,

```

```

104         interval_end=end,
105         source_label=_fmt_source_label(start),
106         template_interval=_fmt_template_interval(day, start, end),
107     )
108 )
109 return slots
110
111
112 def parse_input_time_label(label: str, day: date) -> datetime:
113     """解析输入表时间标签为该标签所代表槽位的区间起点（datetime）。
114
115     - '00:10' ... '23:50' → 当天该时刻；
116     - '0:00+1' → 次日 0:00（= 当天 24:00，槽 143 的起点）。
117     带秒的写法（官方表 '00:10:00' 与 '23:50' 混用）也接受。
118     """
119     s = str(label).strip()
120     cross = "+1" in s
121     s = s.replace("+1", "").strip()
122     if s == "24:00":
123         s = "0:00"
124     hh, mm = s.split(":")[:2]
125     h = int(hh)
126     m = int(mm)
127     if not (0 <= h <= 23 and 0 <= m <= 59):
128         raise ValueError(f"无法解析时间标签: {label!r}")
129     base = datetime.combine(day, datetime.min.time())
130     if cross:
131         return base + timedelta(days=1)
132     return base + timedelta(hours=h, minutes=m)
133
134
135 def slot_id_from_input_label(label: str, day: date) -> int:
136     """输入标签 → slot_id（起点对齐口径，无平移）。
137
138     标签 '0:00+1'（= 24:00）是槽 143 的区间起点 → slot_id 143。
139     '00:10' → slot 0。官方输入表没有 '00:00' 列，遇之报错。
140     """
141     start = parse_input_time_label(label, day)
142     base = datetime.combine(day, datetime.min.time())
143     minutes = (start - base).total_seconds() / 60
144     k = int(round(minutes / SLOT_MINUTES)) - 1
145     if not 0 <= k <= N_SLOTS - 1:
146         raise ValueError(f"标签 {label!r} 不是官方输入标签（槽位 {k}）")
147     return k
148
149
150 def template_column_label(k: int, day: date) -> str:
151     """slot_id k 的模板区间标签。"""
152     return build_day_slots(day)[k].template_interval

```

```

1 """统一 LP 内核：确定性购电 + 储能调度（四问共用）。
2
3 冻结口径（DECISIONS.md D002）：
4 - P0-3  $\eta_c = \eta_d = 0.9$ ，母线侧计量：
5   母线侧能量平衡  $g + v_{\text{used}} + d = 1 + c$ （ $v_{\text{used}}$  = 光伏消纳量）；
6   电池状态  $E_{\{t+1\}} = E_t + \eta_c \cdot c_t - d_t / \eta_d$ 。

```

```

7 - P0-4 允许弃光（仅限光伏）、禁止反送。
8 - P0-2/6 SOC  $\in [1200, 10800]$ , E_0 由调用方给定。
9 - 功率上限 5000 kW  $\rightarrow$  每格电量  $\leq 5000 \cdot \Delta t = 833.333... \text{ kWh}$ 。
10
11 TASK_C_probe_model_corrections.md §3:
12 - 弃光只能来自光伏:  $\text{pv\_used} \leq \text{pv\_available}$ ,  $\text{pv\_curtail} = \text{pv\_available} - \text{pv\_used}$ ;
13   严禁通过增大  $\text{pv\_curtail}$  处理多买的外网电。
14 - Q1 语义:  $\text{grid\_contract} = \text{grid\_delivered}$ ,  $\text{grid\_unused} = 0$ ,  $\text{grid\_emergency} = 0$ ;
15   目标只算  $\text{sum}(\text{price} \cdot \text{grid\_contract})$ 。Q2 的紧急/未使用合同电规则不混入。
16
17 求解器是配置项: scipy HiGS dual simplex 主引擎 + IPM 对拍。
18 字典序吞吐量最小化: 先最小化费用, 再在费用不劣于 tol 的解中最小化
19  $\text{sum}(\text{c}+\text{d})$  (不引入  $\epsilon$  惩罚改变主目标)。
20 """
21
22 from __future__ import annotations
23
24 from dataclasses import dataclass, field
25
26 import numpy as np
27 from scipy.optimize import linprog
28
29 from core.params import BatteryParams, DEFAULT_BATTERY
30
31 EPS = 1e-9
32
33 # 兼容现有调用; 新代码应通过 BatteryParams 显式传参。
34 ETA_C = DEFAULT_BATTERY.eta_charge
35 ETA_D = DEFAULT_BATTERY.eta_discharge
36 SOC_MIN = DEFAULT_BATTERY.soc_min
37 SOC_MAX = DEFAULT_BATTERY.soc_max
38 P_MAX = DEFAULT_BATTERY.power_max
39 DT = DEFAULT_BATTERY.delta_t
40 C_MAX = DEFAULT_BATTERY.energy_limit
41 LEX_TOL = 1e-7
42
43
44 @dataclass
45 class LpInputs:
46     """单日/窗口 LP 输入 (母线侧, kWh 或 kWh/格)。"""
47
48     price: np.ndarray # 电价 元/kWh
49     load: np.ndarray # 负荷 每格 kWh
50     pv_available: np.ndarray # 光伏可用量 每格 kWh
51     soc0: float # 初始 SOC kWh
52     soc_final: float | None = None # 终端 SOC 约束值; None = 自由
53     soc_final_slot: int | None = None # 终端约束作用的槽位下标:
54         # None  $\rightarrow$  全部  $n$  槽之后 ( $E_n = \text{soc\_final}$ , D002 P0-2 冻结口径);
55         #  $k \rightarrow$  前  $k+1$  槽之后 ( $E_{\{k+1\}} = \text{soc\_final}$ , 用于敏感性对照  $E_{143}=E_0$ )
56     terminal_value: float = 0.0 # 终端库存价值 元/kWh (0 时为 Q1/D002 原型)
57     params: BatteryParams = field(default_factory=lambda: DEFAULT_BATTERY)
58
59
60 @dataclass
61 class LpSolution:
62     grid: np.ndarray # 购电 (母线侧) =  $\text{grid\_contract} = \text{grid\_delivered}$  (Q1 语义)

```

```

63 pv_used: np.ndarray
64 pv_curtail: np.ndarray
65 charge: np.ndarray      # c_t 母线侧充电
66 discharge: np.ndarray   # d_t 母线侧放电
67 soc: np.ndarray         # E_t, 长度 n+1 (E[0]=soc0)
68 cost: float             # sum(price * grid), 纯购电费
69 primary_optimum: float  # 第一阶段真实最优目标值 (含终端价值项)
70 objective: float        # 最终选择的真实目标值 (购电费 - 终端价值)
71 status: int
72 message: str
73 solver: str
74
75
76 def solve_lp(inp: LpInputs, solver: str = "highs-ds") -> LpSolution:
77     """求解单日/窗口确定性 LP (含字典序吞吐量最小化)。
78
79     solver: 'highs-ds' (dual simplex) | 'highs-ipm' (内点法)。
80     变量布局: g[0..n), v[0..n), c[0..n), d[0..n), E 由状态方程消去。
81     """
82     n = len(inp.price)
83     assert inp.load.shape == (n,) and inp.pv_available.shape == (n,)
84
85     params = inp.params
86
87     # 变量: g, v, c, d 各 n 个; 弃光由 pv_available - v 推导。
88     def idx(name: str, t: int) -> int:
89         return {"g": 0, "v": 1, "c": 2, "d": 3}[name] * n + t
90
91     nv = 4 * n
92     c_obj = np.zeros(nv)  # 正常购电费用 (用于报告, 永远是最小化目标的一部分)
93     for t in range(n):
94         c_obj[idx("g", t)] = inp.price[t]
95
96     # 终端库存价值: E_n = E_0 + Σ(η_c * c_s - d_s / η_d), 按 terminal_value 计价。
97     # 该项只影响优化目标; 报告的 cost 仍为纯购电费。terminal_value=0 时与原型完全一致。
98     c_opt = c_obj.copy()
99     if inp.terminal_value != 0.0:
100         for t in range(n):
101             c_opt[idx("c", t)] -= inp.terminal_value * params.eta_charge
102             c_opt[idx("d", t)] += inp.terminal_value / params.eta_discharge
103
104     A_ub = []
105     b_ub = []
106     A_eq = []
107     b_eq = []
108
109     # 母线侧能量平衡: g + v + d - l - c = 0
110     for t in range(n):
111         row = np.zeros(nv)
112         row[idx("g", t)] = 1.0
113         row[idx("v", t)] = 1.0
114         row[idx("d", t)] = 1.0
115         row[idx("c", t)] = -1.0
116         A_eq.append(row)
117         b_eq.append(inp.load[t])
118

```

```

119 # 光伏消纳上限（弃光只能来自光伏）： $v \leq pv\_available$ 
120 for t in range(n):
121     row = np.zeros(nv)
122     row[idx("v", t)] = 1.0
123     A_ub.append(row)
124     b_ub.append(inp.pv_available[t])
125
126 # 充放电功率上限： $c \leq C\_MAX$ ,  $d \leq C\_MAX$ 
127 for t in range(n):
128     row = np.zeros(nv)
129     row[idx("c", t)] = 1.0
130     A_ub.append(row)
131     b_ub.append(params.energy_limit)
132     row = np.zeros(nv)
133     row[idx("d", t)] = 1.0
134     A_ub.append(row)
135     b_ub.append(params.energy_limit)
136
137 # SOC 链： $E_{t+1} = E_t + \eta_c \cdot c_t - d_t / \eta_d \in [SOC\_MIN, SOC\_MAX]$ 
138 #  $E_{t+1} = E_0 + \sum_{s \leq t} (\eta_c \cdot c_s - d_s / \eta_d)$ 
139 # 上界： $\sum_{s \leq t} (\eta_c \cdot c_s - d_s / \eta_d) \leq SOC\_MAX - E_0$ 
140 # 下界： $\sum_{s \leq t} (\eta_c \cdot c_s - d_s / \eta_d) \geq SOC\_MIN - E_0$ 
141 for t in range(n):
142     row = np.zeros(nv)
143     for s in range(t + 1):
144         row[idx("c", s)] = params.eta_charge
145         row[idx("d", s)] = -1.0 / params.eta_discharge
146     A_ub.append(row.copy())
147     b_ub.append(params.soc_max - inp.soc0)
148     A_ub.append(-row)
149     b_ub.append(-(params.soc_min - inp.soc0))
150
151 # 终端 SOC 约束（Q1:  $E_n = E_0$ ; soc_final_slot 控制约束作用的槽位）
152 if inp.soc_final is not None:
153     horizon = inp.soc_final_slot if inp.soc_final_slot is not None else n - 1
154     row = np.zeros(nv)
155     for s in range(horizon + 1):
156         row[idx("c", s)] = params.eta_charge
157         row[idx("d", s)] = -1.0 / params.eta_discharge
158     A_eq.append(row)
159     b_eq.append(inp.soc_final - inp.soc0)
160
161 bounds = [(0, None)] * nv # 全部非负（ $g \geq 0$  禁止反送； $v, c, d \geq 0$ ）
162
163 res = linprog(
164     c_obj,
165     A_ub=np.array(A_ub),
166     b_ub=np.array(b_ub),
167     A_eq=np.array(A_eq),
168     b_eq=np.array(b_eq),
169     bounds=bounds,
170     method=solver,
171 )
172
173 primary_optimum = float(res.fun) if res.success else np.nan
174

```

```

175 # 字典序：真实目标（购电费 - 终端价值）不劣于 tol 的解中最小化吞吐量  $\Sigma(c+d)$ 。
176 # 费用上界用真实目标  $c_{opt}$  表达：报告费用仍为纯购电费  $c_{obj} \cdot x$ 。
177 if res.success:
178     c2 = np.zeros(nv)
179     for t in range(n):
180         c2[idx("c", t)] = 1.0
181         c2[idx("d", t)] = 1.0
182     A_ub2 = np.array(A_ub + [c_opt])
183     b_ub2 = np.array(list(b_ub) + [primary_optimum + LEX_TOL])
184     res2 = linprog(
185         c2,
186         A_ub=A_ub2,
187         b_ub=b_ub2,
188         A_eq=np.array(A_eq),
189         b_eq=np.array(b_eq),
190         bounds=bounds,
191         method=solver,
192     )
193     if res2.success:
194         res = res2
195         res.fun = float(c_opt @ res.x) # 报告真实目标值
196
197 if not res.success:
198     return LpSolution(
199         grid=np.full(n, np.nan),
200         pv_used=np.full(n, np.nan),
201         pv_curtail=np.full(n, np.nan),
202         charge=np.full(n, np.nan),
203         discharge=np.full(n, np.nan),
204         soc=np.full(n + 1, np.nan),
205         cost=np.nan,
206         primary_optimum=primary_optimum,
207         status=res.status,
208         message=res.message,
209         solver=solver,
210     )
211
212 x = res.x
213 grid = np.array([x[idx("g", t)] for t in range(n)])
214 pv_used = np.array([x[idx("v", t)] for t in range(n)])
215 pv_curtail = inp.pv_available - pv_used
216 charge = np.array([x[idx("c", t)] for t in range(n)])
217 discharge = np.array([x[idx("d", t)] for t in range(n)])
218
219 soc = np.zeros(n + 1)
220 soc[0] = inp.soc0
221 for t in range(n):
222     soc[t + 1] = (
223         soc[t]
224         + params.eta_charge * charge[t]
225         - discharge[t] / params.eta_discharge
226     )
227
228 return LpSolution(
229     grid=grid,
230     pv_used=pv_used,

```

```

231     pv_curtail=pv_curtail,
232     charge=charge,
233     discharge=discharge,
234     soc=soc,
235     cost=float(c_obj @ x),          # 纯购电费，与终端价值无关
236     objective=float(c_opt @ x),     # 优化器真实目标（购电费 - 终端价值）
237     primary_optimum=primary_optimum,
238     status=res.status,
239     message=res.message,
240     solver=solver,
241 )
242
243
244 def no_storage_schedule(
245     price: np.ndarray,
246     load: np.ndarray,
247     pv_available: np.ndarray,
248     soc0: float,
249 ) -> dict[str, np.ndarray | float]:
250     """同初末 SOC 的可行基线：完全不使用电池。"""
251     pv_used = np.minimum(load, pv_available)
252     pv_curtail = pv_available - pv_used
253     grid = load - pv_used
254     n = len(load)
255     return {
256         "cost": float(np.sum(price * grid)),
257         "grid": grid,
258         "pv_used": pv_used,
259         "pv_curtail": pv_curtail,
260         "charge": np.zeros(n),
261         "discharge": np.zeros(n),
262         "soc": np.full(n + 1, soc0, dtype=float),
263     }

```

2.2 计划执行器与版本账本

```

1  """统一执行器与计划版本账本。
2
3  TASK_C_probe_model_corrections.md §3.3/§5:
4  - 统一结果记录字段: grid_contract, grid_delivered, grid_unused,
5    grid_emergency, pv_available, pv_used, pv_curtail, charge, discharge,
6    soc_start, soc_end。
7  - 计划更新必须追加新版本，不能覆盖旧版本（immutable plan records）:
8    target_slot, issue_time, plan_version, previous_version, quantity,
9    increase_from_previous, decrease_from_previous。
10
11  Q1 单版本执行保持原语义。Q2 按 D007 落实合同余电、紧急购电与计划
12  放电削减；Q3 多次结算由独立版本账本处理。
13  """
14
15  from __future__ import annotations
16
17  from dataclasses import dataclass
18  from datetime import datetime

```

```

19
20 import numpy as np
21 from core.params import BatteryParams, DEFAULT_BATTERY
22
23
24 @dataclass(frozen=True)
25 class PlanRecord:
26     """一个时段的购电计划量（单条不可变记录）。"""
27
28     target_slot: int
29     target_date: str          # ISO 日期，属于该槽位的自然日
30     issue_time: datetime      # 计划制定时刻（decision_time）
31     plan_version: int         # 0 = 0:00 计划； Q3: 1=6:00, 2=12:00, 3=18:00
32     previous_version: int | None
33     quantity: float          # 计划购电量 kWh（母线侧）
34     increase_from_previous: float | None = None
35     decrease_from_previous: float | None = None
36
37
38 @dataclass
39 class ExecRecord:
40     """单时段执行与结算明细（Q1 语义：contract = delivered, unused = 0）。"""
41
42     slot: int
43     interval_start: datetime
44     interval_end: datetime
45     price: float
46     load: float
47     pv_available: float
48     pv_used: float
49     pv_curtail: float
50     grid_contract: float
51     grid_delivered: float
52     grid_unused: float
53     grid_emergency: float
54     charge: float
55     discharge: float
56     soc_start: float
57     soc_end: float
58     residual: float          # 能量平衡残差（应为 ~0）
59     normal_cost: float        # Q1/Q2: price * grid_contract
60     emergency_cost: float     # 5 倍紧急购电（Q1 恒 0）
61     discharge_planned: float | None = None
62     discharge_shortfall: float = 0.0
63     discharge_clipped: bool = False
64     charge_planned: float | None = None
65     charge_shortfall: float = 0.0
66     charge_clipped: bool = False
67
68
69 def execute_q1(
70     grid_contract: np.ndarray,
71     price: np.ndarray,
72     load: np.ndarray,
73     pv_available: np.ndarray,
74     pv_used: np.ndarray,

```

```

75     pv_curtail: np.ndarray,
76     charge: np.ndarray,
77     discharge: np.ndarray,
78     soc0: float,
79     starts: list[datetime],
80     ends: list[datetime],
81     plan_issue_time: datetime,
82     params: BatteryParams = DEFAULT_BATTERY,
83 ) -> list[ExecRecord]:
84     """Q1 执行: 按 0:00 计划执行, 实际值揭示后不重调 (D002 P0-7 的 Q1 版)。
85
86     计划 = 确定性输入 (附件 1 预测值), 实际负荷/PV 即预测值本身;
87     结果记录 contract = delivered, unused = 0, emergency = 0
88     (TASK_C_probe_model_corrections.md §3.2)。
89     """
90     n = len(grid_contract)
91     arrays = {
92         "price": price,
93         "load": load,
94         "pv_available": pv_available,
95         "pv_used": pv_used,
96         "pv_curtail": pv_curtail,
97         "charge": charge,
98         "discharge": discharge,
99     }
100     for name, values in arrays.items():
101         if len(values) != n:
102             raise ValueError(f"{name} 长度 {len(values)} != 计划长度 {n}")
103     if len(starts) != n or len(ends) != n:
104         raise ValueError("槽位起止时间长度与计划不一致")
105
106     E = soc0
107     recs = []
108     for t in range(n):
109         if pv_used[t] < -1e-9 or pv_curtail[t] < -1e-9:
110             raise ValueError(f"slot {t}: 光伏消纳/弃光不得为负")
111         if pv_used[t] > pv_available[t] + 1e-9:
112             raise ValueError(f"slot {t}: 光伏消纳超过可用量")
113         if abs(pv_used[t] + pv_curtail[t] - pv_available[t]) > 1e-8:
114             raise ValueError(f"slot {t}: 光伏消纳与弃光不守恒")
115
116         E_next = (
117             E
118             + params.eta_charge * charge[t]
119             - discharge[t] / params.eta_discharge
120         )
121         residual = (
122             grid_contract[t] + pv_used[t] + discharge[t] - load[t] - charge[t]
123         )
124         recs.append(
125             ExecRecord(
126                 slot=t,
127                 interval_start=starts[t],
128                 interval_end=ends[t],
129                 price=price[t],
130                 load=load[t],

```

```

131         pv_available=pv_available[t],
132         pv_used=pv_used[t],
133         pv_curtail=pv_curtail[t],
134         grid_contract=grid_contract[t],
135         grid_delivered=grid_contract[t],
136         grid_unused=0.0,
137         grid_emergency=0.0,
138         charge=charge[t],
139         discharge=discharge[t],
140         soc_start=E,
141         soc_end=E_next,
142         residual=residual,
143         normal_cost=price[t] * grid_contract[t],
144         emergency_cost=0.0,
145     )
146 )
147 E = E_next
148 return recs
149
150
151 def execute_q2(
152     grid_contract: np.ndarray,
153     price: np.ndarray,
154     load_actual: np.ndarray,
155     pv_available: np.ndarray,
156     charge_planned: np.ndarray,
157     discharge_planned: np.ndarray,
158     soc0: float,
159     starts: list[datetime],
160     ends: list[datetime],
161     plan_issue_time: datetime,
162     params: BatteryParams = DEFAULT_BATTERY,
163     slot_ids: list[int] | None = None,
164 ) -> list[ExecRecord]:
165     """按 D002/D007 顺序回放第二问的实际运行。
166
167     电池动作尽量按计划执行；为保证库存物理边界，必要时只削减所需的
168     充电或放电量。需求分配顺序为计划放电、合同电、实际光伏，
169     仍不足时紧急购电；这等于过剩时先弃光、再形成未用合同电、最后
170     削减计划放电。所有安全削减均显式记入执行记录。
171     """
172     n = len(grid_contract)
173     arrays = {
174         "price": price,
175         "load_actual": load_actual,
176         "pv_available": pv_available,
177         "charge_planned": charge_planned,
178         "discharge_planned": discharge_planned,
179     }
180     for name, values in arrays.items():
181         if len(values) != n:
182             raise ValueError(f"{name} 长度 {len(values)} != 计划长度 {n}")
183         if np.any(np.asarray(values, dtype=float) < -1e-9):
184             raise ValueError(f"{name} 不得为负")
185     if len(starts) != n or len(ends) != n:
186         raise ValueError("槽位起止时间长度与计划不一致")

```

```

187     if slot_ids is None:
188         slot_ids = list(range(n))
189     if len(slot_ids) != n or len(set(slot_ids)) != n:
190         raise ValueError("slot_ids 必须与计划等长且不得重复")
191
192     E = float(soc0)
193     recs: list[ExecRecord] = []
194     for t in range(n):
195         if charge_planned[t] > 1e-9 and discharge_planned[t] > 1e-9:
196             raise ValueError(f"slot {t}: 计划不得同时充放电")
197
198         charge_room = max((params.soc_max - E) / params.eta_charge, 0.0)
199         charge_actual = min(float(charge_planned[t]), charge_room)
200         charge_shortfall = float(charge_planned[t]) - charge_actual
201
202         demand = float(load_actual[t] + charge_actual)
203         discharge_room = max((E - params.soc_min) * params.eta_discharge, 0.0)
204         discharge_actual = min(float(discharge_planned[t]), demand, discharge_room)
205         remaining = demand - discharge_actual
206
207         grid_delivered = min(float(grid_contract[t]), remaining)
208         grid_unused = float(grid_contract[t]) - grid_delivered
209         remaining -= grid_delivered
210
211         pv_used = min(float(pv_available[t]), remaining)
212         pv_curtail = float(pv_available[t]) - pv_used
213         remaining -= pv_used
214         grid_emergency = max(remaining, 0.0)
215
216         discharge_shortfall = float(discharge_planned[t]) - discharge_actual
217         E_next = (
218             E
219             + params.eta_charge * charge_actual
220             - discharge_actual / params.eta_discharge
221         )
222         if E_next < params.soc_min - 1e-6 or E_next > params.soc_max + 1e-6:
223             raise AssertionError(f"slot {t}: 安全削减后SOC仍越界 {E_next:.6f}")
224         residual = (
225             grid_delivered + grid_emergency + pv_used + discharge_actual
226             - float(load_actual[t]) - charge_actual
227         )
228         recs.append(
229             ExecRecord(
230                 slot=slot_ids[t],
231                 interval_start=starts[t],
232                 interval_end=ends[t],
233                 price=float(price[t]),
234                 load=float(load_actual[t]),
235                 pv_available=float(pv_available[t]),
236                 pv_used=pv_used,
237                 pv_curtail=pv_curtail,
238                 grid_contract=float(grid_contract[t]),
239                 grid_delivered=grid_delivered,
240                 grid_unused=grid_unused,
241                 grid_emergency=grid_emergency,
242                 charge=charge_actual,

```

```

243         discharge=discharge_actual,
244         soc_start=E,
245         soc_end=E_next,
246         residual=residual,
247         normal_cost=float(price[t] * grid_contract[t]),
248         emergency_cost=float(5.0 * price[t] * grid_emergency),
249         discharge_planned=float(discharge_planned[t]),
250         discharge_shortfall=discharge_shortfall,
251         discharge_clipped=discharge_shortfall > 1e-9,
252         charge_planned=float(charge_planned[t]),
253         charge_shortfall=charge_shortfall,
254         charge_clipped=charge_shortfall > 1e-9,
255     )
256 )
257 E = E_next
258 return recs
259
260
261 def estimate_bridge_soc(
262     soc_at_midnight: float,
263     charge_planned: float,
264     discharge_planned: float,
265     params: BatteryParams = DEFAULT_BATTERY,
266 ) -> float:
267     """在00:00估计旧计划最后十分钟执行后的00:10库存。
268
269     只使用00:00已经知道的库存和前一天已经安排的电池动作，不读取该十
270     分钟的未来实际负荷或光伏。容量边界采用D008安全削减；因实际需求
271     造成的放电削减要到00:10执行结束后才知道，由新计划执行器吸收偏差。
272     """
273     values = (soc_at_midnight, charge_planned, discharge_planned)
274     if any(not np.isfinite(value) for value in values):
275         raise ValueError("桥接库存和电池动作必须为有限数")
276     if not params.soc_min - 1e-9 <= soc_at_midnight <= params.soc_max + 1e-9:
277         raise ValueError("00:00实际库存越界")
278     if charge_planned < -1e-9 or discharge_planned < -1e-9:
279         raise ValueError("桥接充放电计划不得为负")
280     if charge_planned > 1e-9 and discharge_planned > 1e-9:
281         raise ValueError("桥接槽不得同时计划充放电")
282
283     charge_actual = min(
284         float(charge_planned),
285         max((params.soc_max - soc_at_midnight) / params.eta_charge, 0.0),
286     )
287     discharge_actual = min(
288         float(discharge_planned),
289         max((soc_at_midnight - params.soc_min) * params.eta_discharge, 0.0),
290     )
291     estimate = (
292         soc_at_midnight
293         + params.eta_charge * charge_actual
294         - discharge_actual / params.eta_discharge
295     )
296     if not params.soc_min - 1e-6 <= estimate <= params.soc_max + 1e-6:
297         raise AssertionError("桥接库存估计越界")
298     return float(estimate)

```

```

299
300
301 def records_to_plan_rows(plan: np.ndarray, day: str, issue_time: datetime,
302                          version: int = 0) -> list[PlanRecord]:
303     """Q1 计划 → 不可变计划记录（单版本）。"""
304     rows = []
305     prev_q = None
306     for t, q in enumerate(plan):
307         rows.append(
308             PlanRecord(
309                 target_slot=t,
310                 target_date=day,
311                 issue_time=issue_time,
312                 plan_version=version,
313                 previous_version=version - 1 if version > 0 else None,
314                 quantity=float(q),
315                 increase_from_previous=None,
316                 decrease_from_previous=None,
317             )
318         )
319     prev_q = q
320     return rows

```

2.3 独立核算器

```

1  """独立执行审计器。
2
3  核算器只共享题意参数，不调用 LP 的状态更新或目标函数实现。它从执行
4  记录重新计算能量、SOC、费用和跨槽连续性。
5  """
6
7  from __future__ import annotations
8
9  from dataclasses import dataclass, field
10 from typing import Iterable
11
12 from core.executor import ExecRecord
13 from core.params import BatteryParams, DEFAULT_BATTERY
14
15
16 @dataclass
17 class AuditResult:
18     max_residual: float
19     soc_min: float
20     soc_max: float
21     soc_bounds_ok: bool
22     power_bounds_ok: bool
23     nonneg_ok: bool
24     simultaneous_charge_discharge: int
25     pv_identity_ok: bool
26     curtail_only_from_pv: bool
27     slots_unique: bool
28     slots_contiguous: bool
29     intervals_contiguous: bool

```

```

30 soc_chain_ok: bool
31 q1_semantics_ok: bool # 兼容字段名: 非Q1时表示合同量账本恒等式通过
32 cost_normal: float
33 cost_emergency: float
34 cost_up: float
35 cost_down: float
36 cost_total: float
37 violations: list[str] = field(default_factory=list)
38
39 @property
40 def ok(self) -> bool:
41     return (
42         self.soc_bounds_ok
43         and self.power_bounds_ok
44         and self.nonneg_ok
45         and self.simultaneous_charge_discharge == 0
46         and self.pv_identity_ok
47         and self.curtail_only_from_pv
48         and self.slots_unique
49         and self.slots_contiguous
50         and self.intervals_contiguous
51         and self.soc_chain_ok
52         and self.q1_semantics_ok
53         and self.max_residual < 1e-9
54         and not self.violations
55     )
56
57
58 def audit(
59     recs: Iterable[ExecRecord],
60     params: BatteryParams = DEFAULT_BATTERY,
61     require_q1_semantics: bool = True,
62 ) -> AuditResult:
63     """独立复算执行记录: 空集、乱序、断槽和 SOC 断链均为硬错误。"""
64     recs = list(recs)
65     if not recs:
66         raise ValueError("审计记录不能为空")
67
68     violations: list[str] = []
69     slots = [r.slot for r in recs]
70     slots_unique = len(set(slots)) == len(slots)
71     slots_contiguous = slots == list(range(slots[0], slots[0] + len(slots)))
72     intervals_contiguous = True
73     soc_chain_ok = True
74
75     if not slots_unique:
76         violations.append("槽位编号重复")
77     if not slots_contiguous:
78         violations.append("槽位必须按升序连续排列")
79     for prev, cur in zip(recs, recs[1:]):
80         if prev.interval_end != cur.interval_start:
81             intervals_contiguous = False
82             violations.append(f"slot {prev.slot}->{cur.slot}: 时间区间不连续")
83         if abs(prev.soc_end - cur.soc_start) > 1e-6:
84             soc_chain_ok = False
85             violations.append(f"slot {prev.slot}->{cur.slot}: SOC 链断裂")

```

```

86
87 max_res = 0.0
88 soc_min_seen = float("inf")
89 soc_max_seen = float("-inf")
90 soc_ok = True
91 power_ok = True
92 nonneg_ok = True
93 pv_id_ok = True
94 curtail_src_ok = True
95 q1_semantics_ok = True
96 n_simul = 0
97
98 for r in recs:
99     residual = (
100         r.grid_delivered
101         + r.grid_emergency
102         + r.pv_used
103         + r.discharge
104         - r.load
105         - r.charge
106     )
107     max_res = max(max_res, abs(residual))
108     if abs(r.residual - residual) > 1e-6:
109         violations.append(f"slot {r.slot}: 记录的能量平衡残差与复算值不符")
110
111     expected_soc_end = (
112         r.soc_start
113         + params.eta_charge * r.charge
114         - r.discharge / params.eta_discharge
115     )
116     if abs(expected_soc_end - r.soc_end) > 1e-6:
117         violations.append(
118             f"slot {r.slot}: SOC 转移不符, 记录 {r.soc_end:.9f} "
119             f"vs 复算 {expected_soc_end:.9f}"
120         )
121
122     soc_min_seen = min(soc_min_seen, r.soc_start, r.soc_end)
123     soc_max_seen = max(soc_max_seen, r.soc_start, r.soc_end)
124     for label, value in (("SOC 起点", r.soc_start), ("SOC 终点", r.soc_end)):
125         if value < params.soc_min - 1e-6 or value > params.soc_max + 1e-6:
126             soc_ok = False
127             violations.append(f"slot {r.slot}: {label}越界 {value:.6f}")
128
129     for name, value in (("charge", r.charge), ("discharge", r.discharge)):
130         if value < -1e-9:
131             nonneg_ok = False
132             violations.append(f"slot {r.slot}: {name} 为负 {value:.9f}")
133         if value > params.energy_limit + 1e-6:
134             power_ok = False
135             violations.append(
136                 f"slot {r.slot}: {name} 超功率上限 "
137                 f"{value:.6f} > {params.energy_limit:.6f}"
138             )
139
140     for name, value in (
141         ("grid_contract", r.grid_contract),

```

```

142         ("grid_delivered", r.grid_delivered),
143         ("grid_unused", r.grid_unused),
144         ("grid_emergency", r.grid_emergency),
145         ("pv_available", r.pv_available),
146         ("pv_used", r.pv_used),
147         ("pv_curtail", r.pv_curtail),
148     ):
149         if value < -1e-9:
150             nonneg_ok = False
151             violations.append(f"slot {r.slot}: {name} 为负 {value:.9f}")
152
153     if r.charge > 1e-6 and r.discharge > 1e-6:
154         n_simul += 1
155
156     if r.discharge_planned is not None:
157         if r.discharge > r.discharge_planned + 1e-9:
158             violations.append(f"slot {r.slot}: 实际放电超过计划放电")
159         expected_shortfall = r.discharge_planned - r.discharge
160         if abs(r.discharge_shortfall - expected_shortfall) > 1e-6:
161             violations.append(f"slot {r.slot}: 放电缺额复算不符")
162         if r.discharge_clipped != (expected_shortfall > 1e-9):
163             violations.append(f"slot {r.slot}: 放电削减标志不符")
164     if r.charge_planned is not None:
165         if r.charge > r.charge_planned + 1e-9:
166             violations.append(f"slot {r.slot}: 实际充电超过计划充电")
167         expected_shortfall = r.charge_planned - r.charge
168         if abs(r.charge_shortfall - expected_shortfall) > 1e-6:
169             violations.append(f"slot {r.slot}: 充电缺额复算不符")
170         if r.charge_clipped != (expected_shortfall > 1e-9):
171             violations.append(f"slot {r.slot}: 充电削减标志不符")
172
173     if abs(r.pv_used + r.pv_curtail - r.pv_available) > 1e-6:
174         pv_id_ok = False
175         violations.append(f"slot {r.slot}: 光伏消纳与弃光不守恒")
176     if r.pv_used > r.pv_available + 1e-9 or r.pv_curtail > r.pv_available + 1e-9:
177         curtail_src_ok = False
178         violations.append(f"slot {r.slot}: 弃光/消纳超过光伏可用量")
179     if r.pv_available <= 1e-9 and r.pv_curtail > 1e-9:
180         curtail_src_ok = False
181         violations.append(f"slot {r.slot}: 无光伏却出现弃光")
182
183     if require_q1_semantics:
184         if abs(r.grid_contract - r.grid_delivered) > 1e-6:
185             q1_semantics_ok = False
186             violations.append(f"slot {r.slot}: Q1 合同购电不等于实际受电")
187         if abs(r.grid_unused) > 1e-9 or abs(r.grid_emergency) > 1e-9:
188             q1_semantics_ok = False
189             violations.append(f"slot {r.slot}: Q1 出现未用合同电或紧急购电")
190     elif abs(r.grid_contract - r.grid_delivered - r.grid_unused) > 1e-6:
191         q1_semantics_ok = False
192         violations.append(
193             f"slot {r.slot}: 合同量不等于实际交付量与未使用量之和"
194         )
195
196     # Q1/Q2 的正常费用都按计划合同量收取。Q3 多版本调整由独立账本核算。
197     expected_normal = r.price * r.grid_contract

```

```

198     if abs(r.normal_cost - expected_normal) > 1e-6:
199         violations.append(f"slot {r.slot}: 正常购电费用复算不符")
200     expected_emergency = 5.0 * r.price * r.grid_emergency
201     if abs(r.emergency_cost - expected_emergency) > 1e-6:
202         violations.append(f"slot {r.slot}: 紧急购电费用复算不符")
203
204     cost_normal = float(sum(r.price * r.grid_contract for r in recs))
205     cost_emergency = float(sum(5.0 * r.price * r.grid_emergency for r in recs))
206     cost_up = float(sum(getattr(r, "up_cost", 0.0) for r in recs))
207     cost_down = float(sum(getattr(r, "down_cost", 0.0) for r in recs))
208
209     return AuditResult(
210         max_residual=max_res,
211         soc_min=float(soc_min_seen),
212         soc_max=float(soc_max_seen),
213         soc_bounds_ok=soc_ok,
214         power_bounds_ok=power_ok,
215         nonneg_ok=nonneg_ok,
216         simultaneous_charge_discharge=n_simul,
217         pv_identity_ok=pv_id_ok,
218         curtail_only_from_pv=curtail_src_ok,
219         slots_unique=slots_unique,
220         slots_contiguous=slots_contiguous,
221         intervals_contiguous=intervals_contiguous,
222         soc_chain_ok=soc_chain_ok,
223         q1_semantics_ok=q1_semantics_ok,
224         cost_normal=cost_normal,
225         cost_emergency=cost_emergency,
226         cost_up=cost_up,
227         cost_down=cost_down,
228         cost_total=cost_normal + cost_emergency + cost_up + cost_down,
229         violations=violations,
230     )

```

附录 C 四个指定日期的分块充放电结果

表中每格为“充电量/放电量”，单位均为 kWh；按真实墙钟区间归属。完整 10 分钟购电量和紧急购电区间见五份正式结果文件。

问题	日期	0-4 时	4-8 时	8-12 时	12-16 时	16-20 时	20-24 时
二	3 月 20 日	1422.9/0.0	833.3/4408.6	5000.0/4229.0	6270.5/491.5	0.0/4848.0	3818.9/3609.9
二	6 月 21 日	0.0/0.0	538.3/1735.7	5632.1/0.0	4950.8/0.0	0.0/5389.9	8333.3/2985.9
二	9 月 23 日	1370.9/0.0	833.3/6213.5	3355.9/1775.9	6725.4/176.3	0.0/4779.9	8333.3/3729.1
二	12 月 21 日	1240.1/0.0	1666.7/675.0	5000.0/8640.0	8648.8/2415.5	0.0/4940.0	8333.3/3574.0
三	3 月 20 日	1422.9/0.0	833.3/5896.3	3494.5/2741.3	7470.5/244.0	0.0/4848.0	3818.9/3609.9
三	6 月 21 日	0.0/0.0	512.4/1622.7	776.6/0.0	9705.1/0.0	28.8/5414.4	8166.7/2985.9
三	9 月 23 日	1537.6/0.0	833.3/6370.7	4615.6/2269.3	6451.0/324.0	0.0/4779.9	8166.7/3729.1
三	12 月 21 日	1406.7/0.0	1666.0/676.8	5000.0/8373.4	8465.1/2531.0	0.0/4940.0	8166.7/3574.0

表 12 问题二、三指定日期的六个墙钟分块充放电量